

Solving Combinatorial Optimization Problems using QAOA: Encodings, Dealing with Constraints, and Evaluation Measures

Ulrike Stege

Department of Computer Science, University of Victoria



UVIC



QUANTUM BC
research · training · innovation

Thanks to: Priya Angara, Tristan Zaborniak, Hausi Müller, Emily Martins: UVic;
Yuri Alexeev, Danylo Lykov: NVIDIA; Ibrahim Shehzad, Sean Wagner: IBM

PINQ²

IEEE
QUANTUM
WEEK

Students: you
might still get
in as student
volunteer

IEEE Quantum Week 2026

qce.quantum.ieee.org/2026

IEEE Int Conference on Quantum Computing & Engineering
Sep 13-18, 2026 — Metro Toronto Convention Centre

9 World-Class Keynote Speakers
46 Community-building Workshops
48 Workforce-building Tutorials
80+ Sponsors & Exhibitors

925 Technical Paper Submissions to 9 Tracks

Panel Proposal Submission Deadline: June 22

Poster Proposal Submission Deadline: June 29

Student Mentorship & Career Fair

QSEEC Education Conference



IEEE FINANCIAL CO-SPONSORS



IEEE TECHNICAL CO-SPONSORS

Part 1 — Solving COPs quantumly

- Preliminaries
 - Combinatorial optimization problems (COPs)
 - Computational hard COPs
 - Unconstrained vs constrained COPs
 - QAOA—overview
 - Unconstrained binary optimization formulations (UBOs)
- An unconstrained COP (Maximum Profit Cover) for a constrained COP (Minimum Vertex Cover)
- QUBOs for Minimum Vertex Cover and Maximum Profit Cover

Part 2 — SCOOP

- SCOOP framework — handling constraints
 - Another example: Minimum Dominating set and Profit Domination
 - HUBOs
- From UBOs to cost Hamiltonians to quantum circuits
- Evaluation measures for COPs solved quantumly
- Solving COPs in practice: what to classical and what to quantum? Learning to offload

Solving COPs is essential



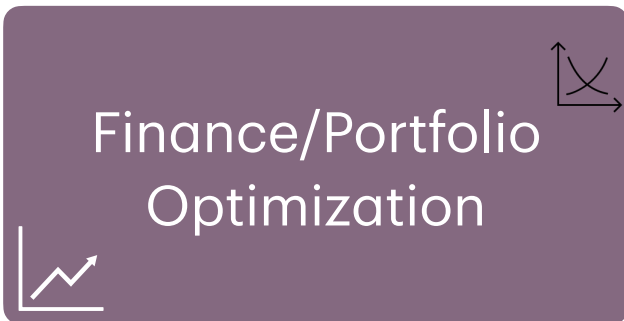
Logistics



Network Design
& Analysis



Scheduling



Finance/Portfolio
Optimization



Machine Learning &
Data Science



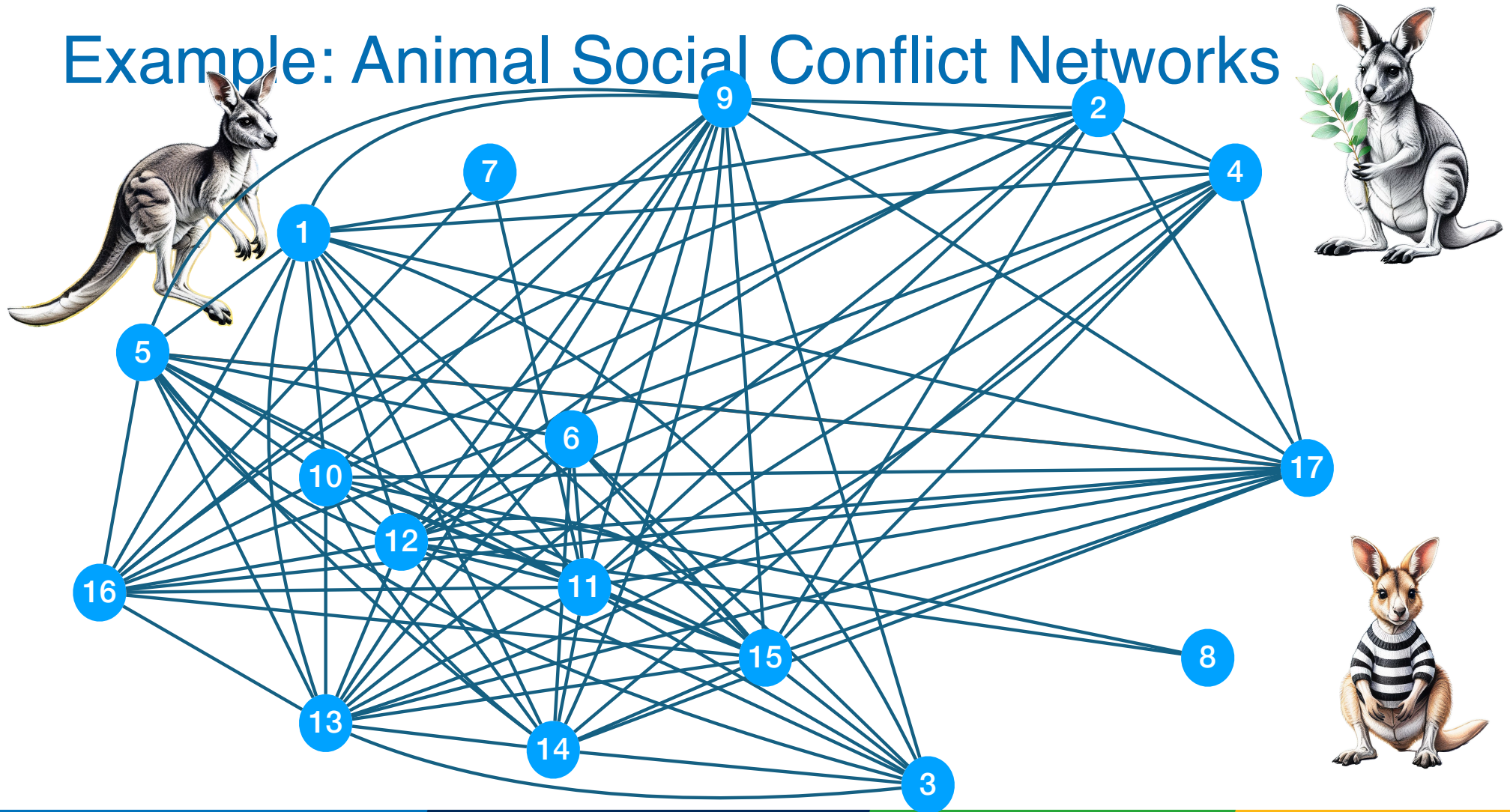
Computational
Biology



What are COPs?

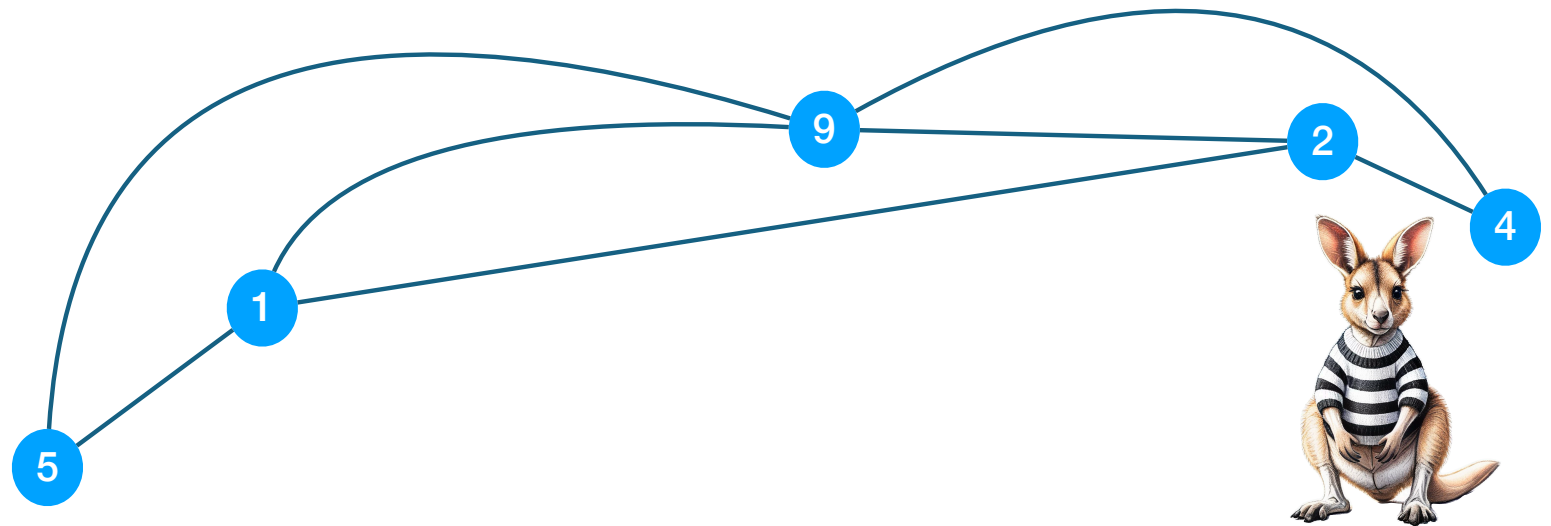
- Solutions of COPs: discrete sets
- Determining solutions typically involves detecting optimal groupings or orderings from a finite set of objects
- These solutions might have to satisfy specific constraints

Example: Animal Social Conflict Networks



Example: Animal Social Conflict Networks

- Which kangaroos need to be replaced to remove all conflicts??

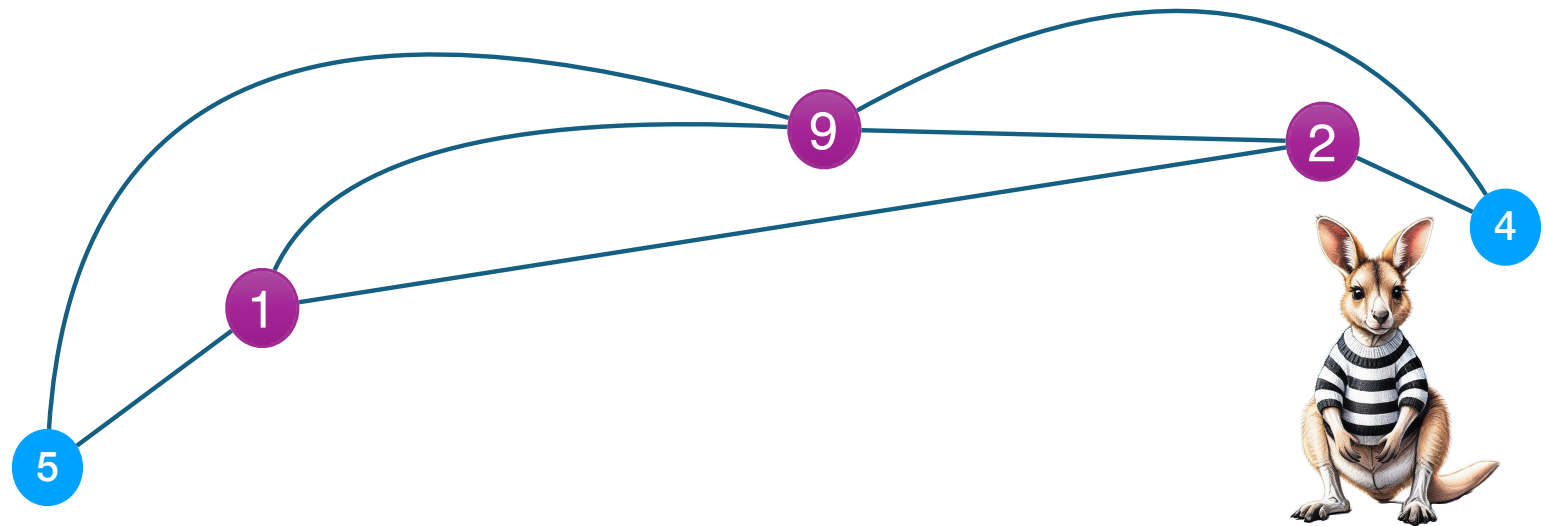


Example: Animal Social Conflict Networks

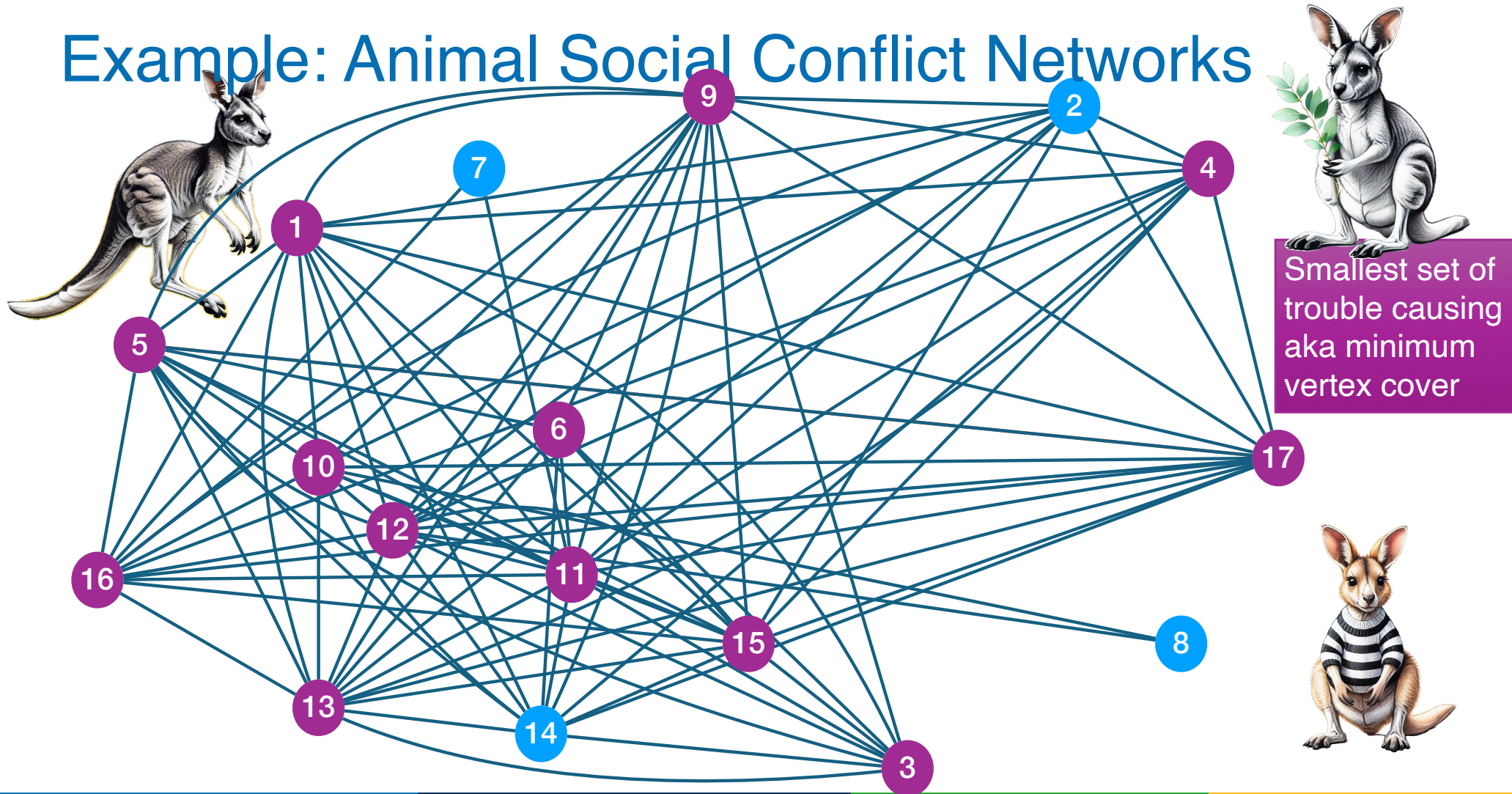
- Which kangaroos need to be replaced to remove all conflicts??



What is a smallest subset of kangaroos that involves/**covers** every interaction?



Example: Animal Social Conflict Networks



Example: Animal Social Networks

- Smallest set of *key-contributors* that connect the network?
 - Minimum vertex cover or small vertex covers



MINIMUM VERTEX COVER

Input: a simple undirected graph $G = (V, E)$

Output: a minimum vertex cover for G , ie a subset $S \subseteq V$ such that every edge in G is covered by a vertex in S and S is minimized



MINIMUM VERTEX COVER is
a combinatorial optimization
problem

MINIMUM VERTEX COVER is
computationally hard

What are COPs?

- Solutions of COPs are discrete (sub)sets
- Determining solutions typically involves detecting optimal groupings or orderings from a finite set of objects
- Many COPs are *computationally hard* to solve:
 - unknown whether there's a polynomial-time algorithm (in terms of input size) that finds a best answer, regardless of the input
 - Known exact classical algorithms do not scale

Combinatorial Optimization Problems

Famous Examples

NP-hard [Garey & Johnson, 1979]

- **Traveling Salesperson problem**



- **Knapsack problem**



- **Set Cover problem**



- **Matching problems**



Depends on the variant

- Minimum Spanning Tree problems



- Maximum Flow problems



- **Independent Set problem**



- **Vertex Cover problem**



- **Clique problem**



- **Dominating Set**



- **Max Cut problem**



Classes P & NP

Decision (Yes/No)-Problem

Class P: solvable in polynomial time on Turing machine/classical computer

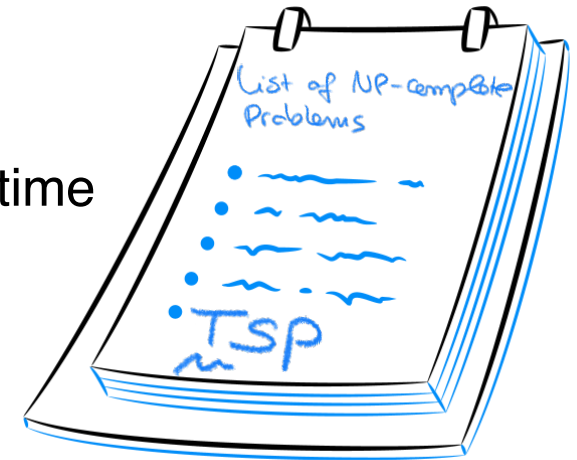
Class NP: yes-answers verifiable in polynomial time on Turing machine

P=NP?

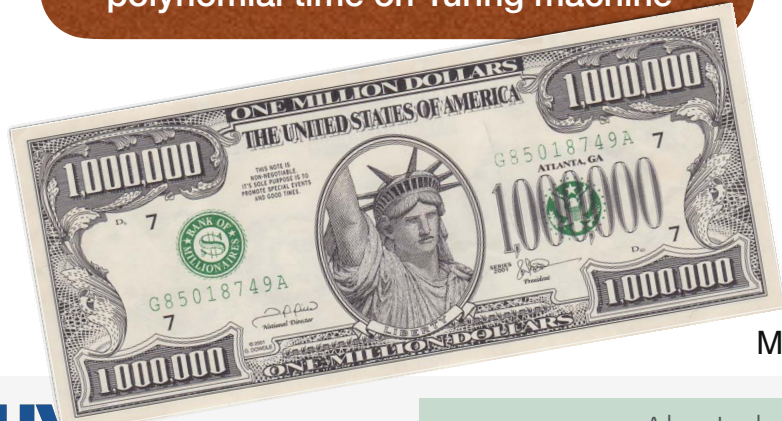
Assumption
 $P \neq NP$

Algorithms running time is measured in size of input

Pairwise polynomial-time reducible



Solving just one of 🙌 in polynomial time implies: every NP-complete problem is solvable in polynomial time $\Rightarrow P = NP$



Millennium Prize Problem by Clay Mathematics Institute

NP-complete problems & Quantum Computing

- Can we solve NP-complete problems in polynomial time using a universal quantum computer/quantum algorithm?
 - Quantum computer scientists so far don't expect this
 - No-one yet been able to show that an NP-complete problem is in BQP
- Does solving an NP-complete problem in polynomial time with a quantum algorithm imply that $P = NP$?
 - No, not unless there is a classical algorithm that in polynomial time simulates any polynomial-time uniform quantum circuit family that output 0 or 1

$P \subseteq BQP$

BQP: Class of problems
Solvable in polynomial time
on a quantum computer

$NP \subseteq BQP?$

Assumption: NP not
a subset of BQP

Classically hard optimization problems

Tackling Intractability

- **NP-hard problem** assumption [Garey & Johnson, 1979]—no polynomial time/efficient algorithm exists that guarantee to find **optimum**
- Efficient algorithms may exist for data inputs with particular properties
- **Near optimal solutions:** Many NP-hard problems are even **hard to approximate**
- Not all NP-hard are equally difficult to solve: Parameterized Complexity [Downey & Fellows, 1999]
 - **FPT:** Parameterizations can exist that permit fixed-parameter approaches; tractable **preprocessing** that **reduces input size**
 - Practicality depends on overall running time of algorithm, choice of parameter and data properties
 - **Hard parameterizations**

Combinatorial Optimization Problems

Parameterized Complexity: FPT [Downey & Fellows, 1999]

- **Traveling Salesperson problem**
in FPT for **treewidth** [Bodlaender et al. 2015]



- **Knapsack problem**

Pseudo-polynomial



- **Set Cover problem**



$W[2]$ -complete for solution size

- **Matching problems**



Depends on the variant

- Minimum Spanning Tree problems



- Maximum Flow problems



- **Independent Set problem**

$W[1]$ -complete for solution size

- **Vertex Cover problem**

In FPT for solution size



- **Clique problem**

$W[1]$ -complete for solution size



- **Dominating Set**

$W[2]$ -complete for solution size



- **Max Cut problem**

In FPT for solution size



Solving COPs

- NP-hard problems: Determining optimum is often infeasible — especially for large instances that are not “nice”
- Applications—near-optimal solutions that are found much faster can be important solutions
 - pick “favourite” from a set of high quality feasible solutions (eg favourite kangaroo must stay)
- **Classical algorithms** that return optimum: typically return **one** best **solution**, and no other best or solutions close to optimum; even true for approx./heuristics
- **Quantum algorithms are probabilistic**: they return high quality solutions, re-runs result in a number of high-quality solutions
 - Algorithms like QAOA can generate distributions over bit strings, with higher probabilities for lower-cost (better) solutions

COPs—a bit more formal

A combinatorial optimization problem $\mathcal{P}$ can usually be written as:
 $\min_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x})$ or $\max_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x})$ where $\mathcal{F} \subseteq \{0,1\}^n$ is the set of **feasible solutions** and $f(\mathbf{x})$ is the objective function of $\mathcal{P}$

$\mathcal{P}$ is **unconstrained** if $\mathcal{F} = \{0,1\}^n$, otherwise $\mathcal{P}$ is **constrained**

MINIMUM VERTEX COVER

Input: a simple undirected graph $G = (V, E)$

Output: a minimum vertex cover for G , ie a subset $S \subseteq V$ such that every edge in G is covered by a vertex in S and S is minimized

Describe solution as binary vector $\mathbf{x}$

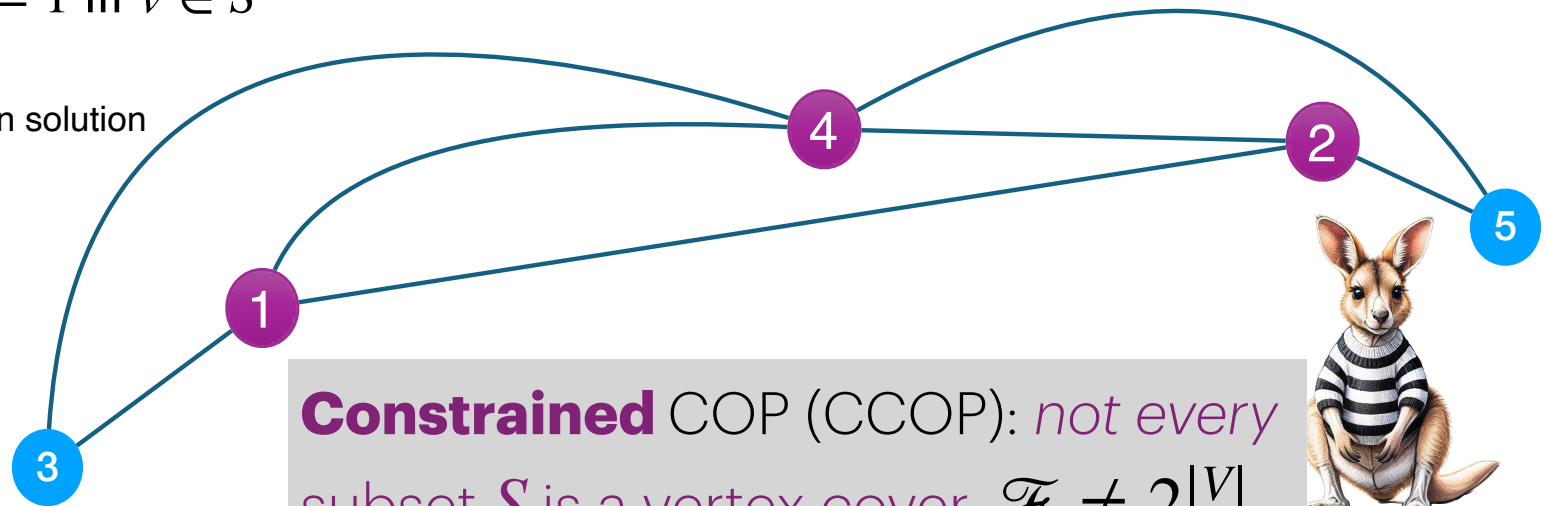
$\mathbf{x} = [1, 1, 0, 1, 0] : x_v = 1 \text{ iff } v \in S$

Objective function:
counts number of vertices in solution

$$\min_{\mathbf{x} \in \{0,1\}^n} \sum_{v \in V} x_v$$



& $\mathbf{x}$ must be feasible
(ie a vertex cover)



Constrained COP (CCOP): *not every* subset S is a vertex cover, $\mathcal{F} \neq 2^{|V|}$



Can Quantum Computing help?

- Quantum Computing has algorithmic tools such as
 - Grover's Adaptive Search (GAS) [Gilliam et al., 2019, arxiv:1912.04088]
 - **Quantum Approximate Optimization Algorithm (QAOA)** [Farhi et al., 2014]
 - **Quantum Annealing (QA)** [McGeoch, 2014]
- These are designed to solve $\min_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x})$ for $\mathcal{F} = \{0,1\}^n$
 - Via **Unconstrained** Optimization Problems (UBOs)—Quadratic UBOs (QUBOs) for QA
- All methods natively solve UBOs, ie **unconstrained** combinatorial optimization problems
- What if $\mathcal{F} \neq \{0,1\}^n$? UBOs with penalty terms, introduce *penalty parameter(s)* that can be hard to determine

CCOP

Unconstrained Binary Optimization (UBO)

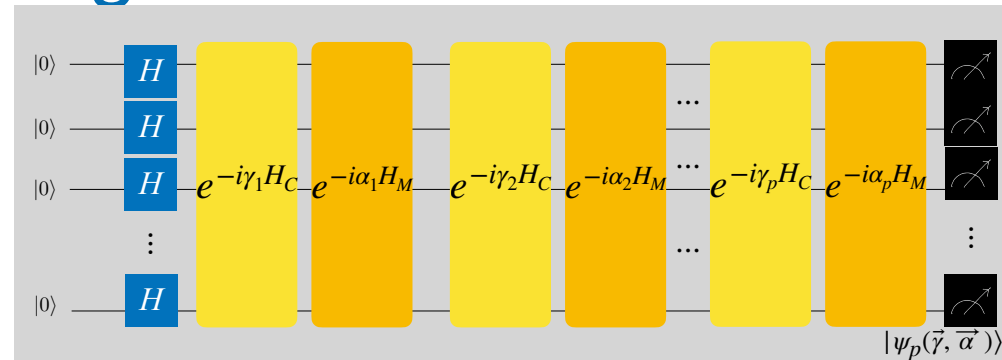
- UBO class of optimization problems
 - Variables are binary: $x_i \in \{0,1\}$
 - No explicit constraints on x_i other than binary
 - Goal: maximize or minimize a given objective function over all possible binary assignments
- Formally $\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x})$ or $\max_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x})$ where:
 - $\mathbf{x} = [x_1, x_2, \dots, x_n]$: vector of **binary variables**
 - $f(\mathbf{x})$: real-valued objective function, usually a polynomial
 - **If at most quadratic**: Quadratic UBO or QUBO
 - **If more than quadratic**: Higher order UBO or HUBO
- Constraints are typically handled by the introduction of penalty terms, tuned using penalty parameters

binary	↔	spin
QUBO form		Ising form
HUBO		Higher order spin

$$\text{QUBO } f_Q(\vec{x}) = \vec{x}^\dagger Q \vec{x}$$

Variational Quantum Algorithms

- QAOA: “layered” variational quantum circuit
- Ansatz — Alternating layers of



$$\langle \psi | H_C | \psi \rangle$$

$$(\vec{\alpha}, \vec{\gamma})$$

Classical
Optimizer

- Cost Operator ($U_C = e^{-i\gamma H_C}$) → encodes problem, optimal = ground state
- Mixer Operator ($U_M = e^{-i\alpha H_M}$) → explores solution space
- Optimization
 - Parameters $(\vec{\alpha}, \vec{\gamma})$ tuned by a classical optimizer

Goal: Minimize expectation value

$$\min_{\vec{\alpha}, \vec{\gamma}} \langle \psi(\vec{\alpha}, \vec{\gamma}) | H_C | \psi(\vec{\alpha}, \vec{\gamma}) \rangle$$



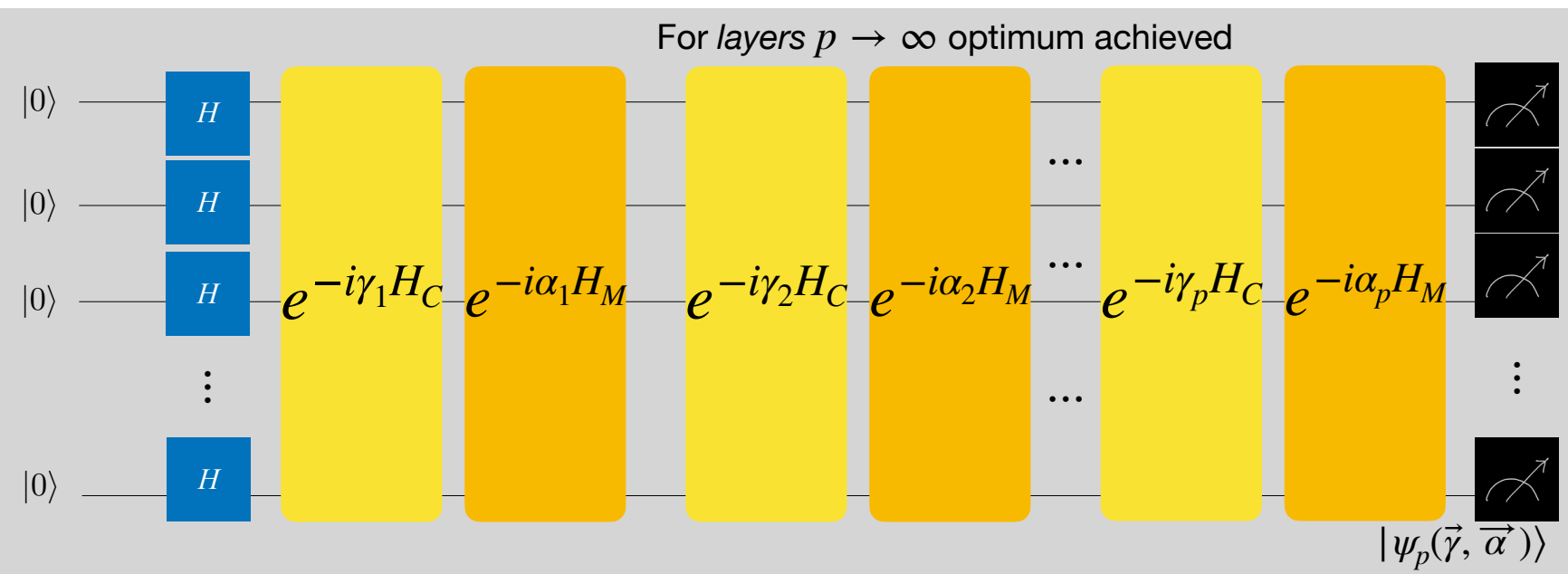
Returns binary solution vector of ground state of H_C

[Farhi, Goldstone, Gutmann, 2014]

Components of QAOA

QAOA: Variational Quantum Algorithm

Alternating Ansatz



H_C : cost Hamiltonian, encodes objective function
 H_M : mixer Hamiltonian

γ_j : parameter “how strong” cost function is applied
 α_j : “push strength” helps keep from getting stuck

Classical
Optimizer

$$\langle \psi | H_C | \psi \rangle$$



$$(\vec{\alpha}, \vec{\gamma})$$

Goal:
 High
 probability /
 good quality
 solution

QAOA: Solving combinatorial optimization problems

- QAOA
 - primarily designed to solve **unconstrained** optimization problems
 - discretized, gate-model-friendly version of adiabatic algorithms
- Compute $|\psi_p(\vec{\gamma}, \vec{\beta})\rangle = e^{-i\alpha_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\alpha_2 H_M} e^{-i\gamma_2 H_C} e^{-i\alpha_1 H_M} e^{-i\gamma_1 H_C} H^{\otimes n} |0\rangle^{\otimes n}$
 - unitary matrices $e^{-i\gamma_i H_C}$ and $e^{-i\alpha_j H_M}$, and cost Hamiltonian H_C and mixer Hamiltonian H_M are Hermitian matrices; parameters are tune-able variational angles $\gamma_i, \alpha_j \in [0, 2\pi[$
 - H_C : Ising or higher order spin form
 - Computation mimics adiabatic evolution from ground state of H_M to ground state of H_C
- Problem most explored using QAOA approach: MAXCUT

Mixer Hamiltonian H_M

- Responsible for “mixing” the state space, avoids solutions getting stuck at local optimum
- Similar to mutation operator in genetic algorithms

- **Most common choice for mixer Hamiltonian:** $H_M = \sum_{j=1}^n X_j$

H_C and H_M

- Both are Hermitian matrices: $H = H^\dagger$
 - Mixer is designed such that H_M is Hermitian
 - Cost Hamiltonians derived from QUBOs also Hermitian
- H_C : diagonal
- H_C and H_M : in general not unitary
- $U_C = e^{-i\gamma H_C}$ and $U_M = e^{-i\alpha H_M}$ are unitary

Given **Hermitian** H , properties of matrix exponential $e^{-i\alpha H}$ include

- Every *eigenvector* $|\phi_k\rangle$ of H is an *eigenvector* of $e^{-i\alpha H}$
- Thus $e^{-i\alpha H} |\phi_k\rangle = e^{-i\alpha\lambda_k} |\phi_k\rangle$ where λ_k is *eigenvalue* of H
- Applying $e^{-i\alpha H}$ to a superposition state $|\psi\rangle$:
 $|\psi\rangle = \sum_k c_k |\phi_k\rangle$ where $c_k = \langle \phi_k | \psi \rangle$ (projection of $|\psi\rangle$ with $|\phi_k\rangle$)

$$\text{Then } |\psi\rangle = \sum_k \langle \phi_k | \psi \rangle \cdot |\phi_k\rangle$$

$$\text{Therefore: } |\psi(\alpha)\rangle = U(\alpha) |\psi\rangle = e^{-i\alpha H} |\psi\rangle = \sum_k \langle \phi_k | \psi \rangle e^{-i\alpha\lambda_k} |\phi_k\rangle$$

Applying $e^{-i\alpha H}$ results in the same ground state and corresponding solution!

What is our goal?

- Take a COP, formulate it as UBO Handling hard constraints
- Derive its cost Hamiltonian H_C
- Determine its ground state by determining the ground state of its unitary (matrix exponential) e^{-itH_C} using, eg QAOA

Main Challenges?

- (1) UBO that models COP accurately
- (2) executing QAOA on hardware

Computationally hard problems in practice

- Many combinatorial problems are NP-hard
- NP-hardness: worst-case intractability
- Small instances/instances with favourable structure can be solved (efficiently) exactly
- Large instances or structurally unfavourable instances remain resistant

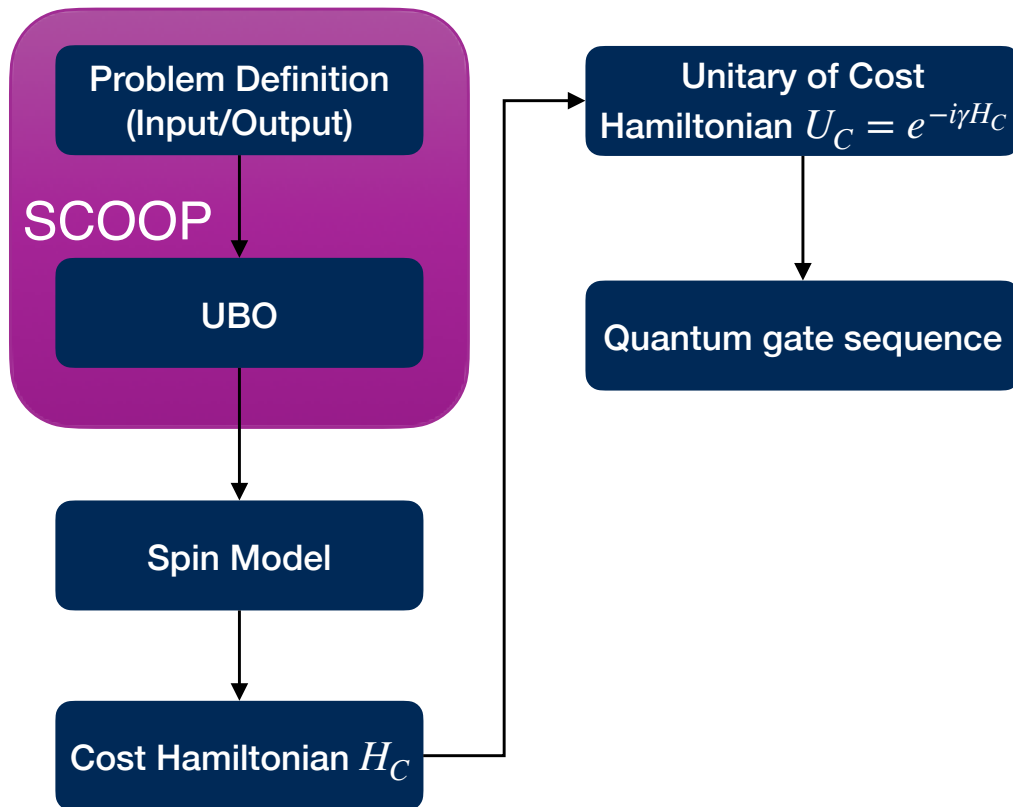
A practical consequence

- *Solve* or *pre-process* what classical methods can do efficiently and exactly
- *Offload* remaining hard instance to quantum hardware
- *Post-process* classically and efficiently

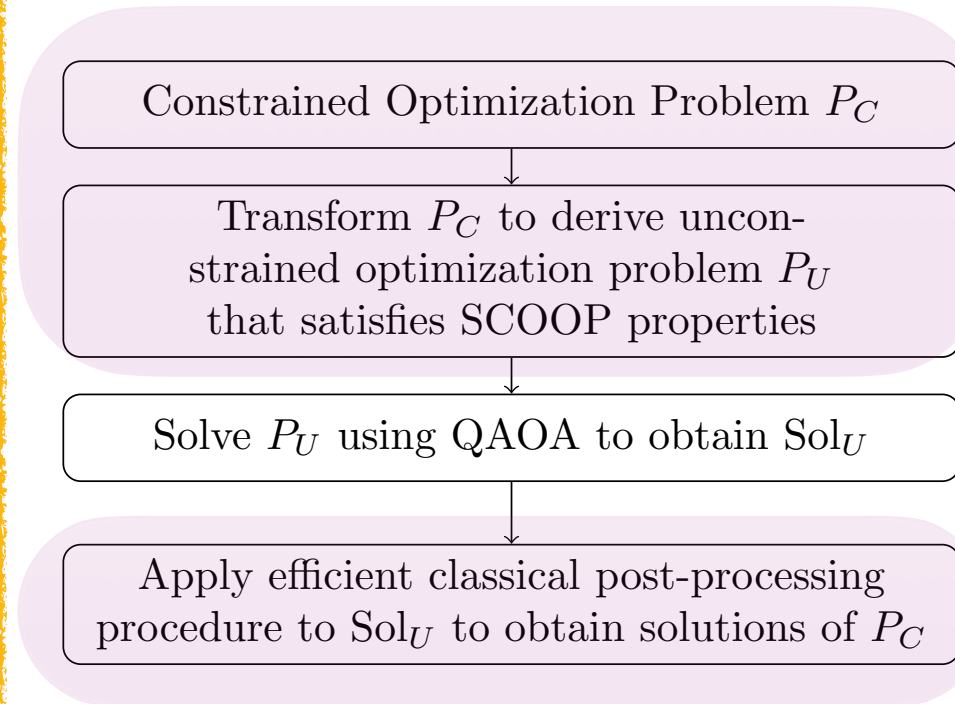
Takeaway

Quantum advantage for combinatorial optimization is likely to emerge from tightly integrated hybrid *classical-quantum* workflows

Encoding Pipeline: From Constrained Combinatorial Optimization Problems to quantum gate sequences

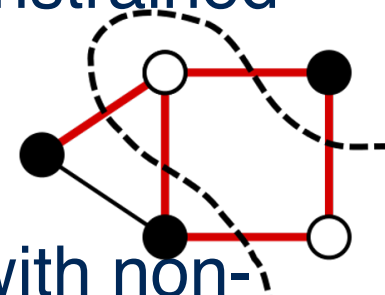


Hybrid Pipeline: Solving constrained COPs using the SCOOP framework



Quantum Approximate Optimization Algorithm

- QAOA widely studied for MAXCUT: an UCOP
- Cost Hamiltonian encoded using Quadratic Unconstrained Binary Optimization (QUBO)
- CCOPs: not all subsets feasible
- Challenge: QAOA produces infeasible solutions with non-zero probability



Tackling CCOPs: A challenge

Real-world applications demand both **feasibility** and **optimality** under multiple constraints.
Encoding constraints and ensuring feasible solutions is a challenge

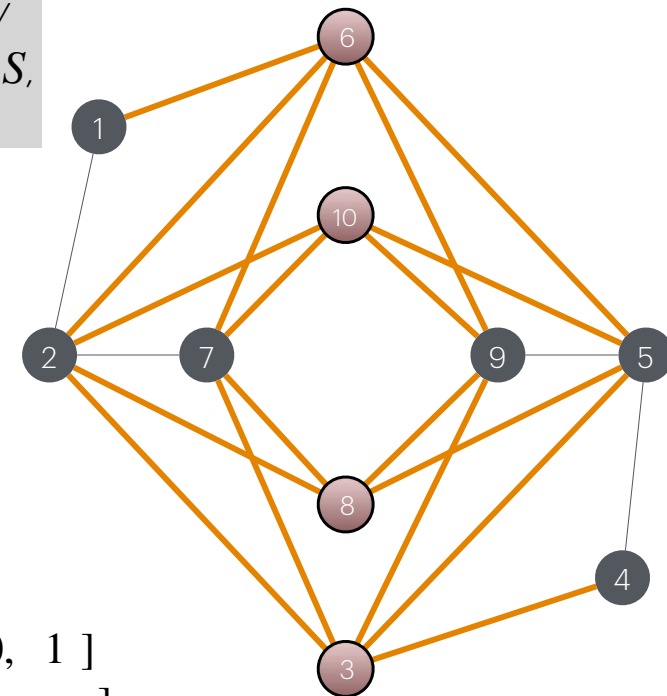
MAXCUT

Unconstrained COP: every subset S , together with $V \setminus S$, forms a cut

Input: a simple undirected graph $G = (V, E)$

Output: $V_1 \subseteq V$ such that the *cut* of G , $\text{cut}(V_1) = \{(u, v) \in E \mid u \in V_1 \text{ and } v \in V_2\}$, is as large as possible. Here $V_2 = V - V_1$.

$$C(x) = |\text{cut}(V_1)|$$



Writing MAXCUT as unconstrained binary optimization (UBO) form:

$$\mathbf{x} = [0, 0, 1, 0, 0, 1, 0, 1, 0, 1] \\ [x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}]$$

$$\max_{\mathbf{x} \in \{0,1\}^n} C(\mathbf{x}) : C(\mathbf{x}) = \sum_{uv \in E} (x_u + x_v - 2x_u x_v)$$

UBO is *quadratic*,
thus QUBO

$$x_u = x_v \Rightarrow x_u + x_v - 2x_u x_v = 0 \\ x_u \neq x_v \Rightarrow x_u + x_v - 2x_u x_v = 1$$

Observed Encoding Gadgets

Choosing **exactly one** element of a set
 $\{i, j\}$ consisting of two elements

$$x_i + x_j - 2x_i x_j$$

MAXCUT as QUBO (continued)

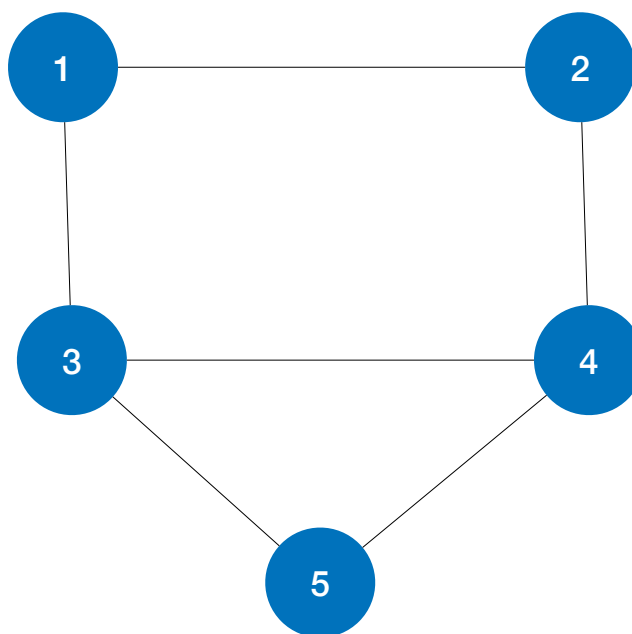
Goal: Determine $\mathbf{x}$ such that $\sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$ maximized

Since we are after *ground states*:

we write equivalently: determine $\mathbf{x}$ such that

$$\sum_{(i,j) \in E} (2x_i x_j - x_i - x_j) \text{ is minimized}$$

MAXCUT Example



Best Solution:

$$\mathbf{x} = [0 \ 1 \ 1 \ 0 \ 0]$$

with $\sum_{(i,j) \in E} (2x_i x_j - x_i - x_j)$

$$= -5$$

Find $\mathbf{x}$ that minimizes:

$$\sum_{(i,j) \in E} (2x_i x_j - x_i - x_j) = \boxed{2x_1 x_2 - x_1 - x_2} + \boxed{2x_1 x_3 - x_1 - x_3} + \boxed{2x_2 x_4 - x_2 - x_4} + \boxed{2x_3 x_4 - x_3 - x_4}$$

$$+ \boxed{2x_3 x_5 - x_3 - x_5} + \boxed{2x_4 x_5 - x_4 - x_5}$$

Solution space of UBOs ...

- ... such as MaxCut can be accurately reflected by UBOs
- For $|V| = n$, every $\mathbf{x} \in \{0,1\}^n$ can be evaluated as a cut (and its cut-size $C(\mathbf{x})$ can be determined)

A Note on Unconstrained Binary Optimization Formulations for CCOPs

When formulating constrained combinatorial optimization problems, constraints are typically *penalized* by increasing the value of the function by detecting violations of the constraint

Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models

Fred Glover¹, Gary Kochenberger², Yu Du²

Abstract

Quantum Bridge Analytics relates generally to methods and systems for hybrid classical-quantum computing, and more particularly is devoted to developing tools for bridging classical and quantum computing to gain the benefits of their alliance in the present and enable enhanced practical application of quantum computing in the future.

This is the first of a two-part tutorial that surveys key elements of Quantum Bridge Analytics and its applications, with an emphasis on supplementing models with numerical illustrations. In Part 1 (the present paper) we focus on the Quadratic Unconstrained Binary Optimization (QUBO)

Ising formulations of many NP problems

Andrew Lucas

Department of Physics, Harvard University, Cambridge, MA, USA 02138

We provide Ising formulations for many NP-complete and NP-hard problems, including all of Karp's 21 NP-complete problems. This collects and extends mappings to the Ising model from partitioning, covering and satisfiability. In each case, the required number of spins is at most cubic in the size of the problem. This work may be useful in designing adiabatic quantum optimization algorithms.

lucas@fas.harvard.edu

January 27, 2014

mech] 24 Jan 2014

A Note on UBOs for CCOPs

- When formulating constrained combinatorial optimization problems, constraints are typically *penalized* by increasing the value of the function by detecting violations of the constraint
- **Note:** Any UBO formulation is unconstrained; penalty parameter(s) must be set
- However: **non-optimal solutions can be infeasible solutions**; there are many infeasible solutions
- Limits success probability to obtain *good* or *near-optimal solutions*

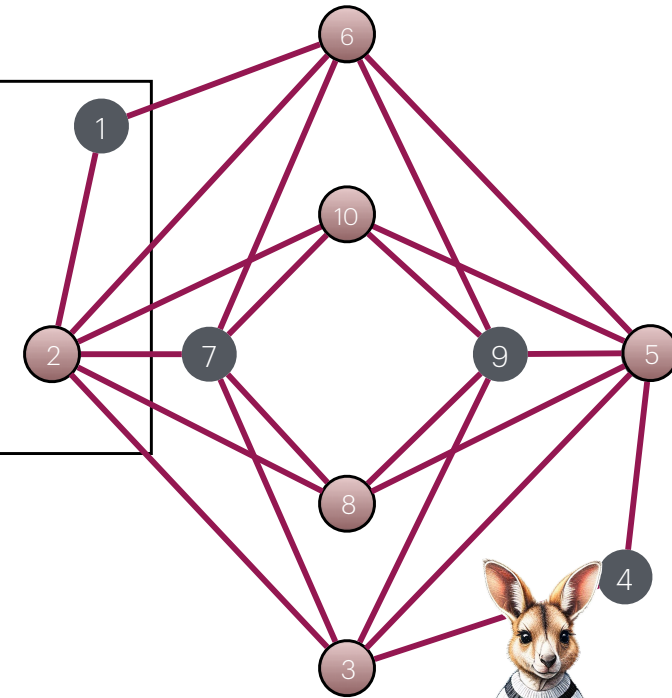
MINIMUM VERTEX COVER: CCOP

MINIMUM VERTEX COVER

Input: a simple undirected graph $G = (V, E)$

Output: a minimum vertex cover for G , ie a subset $S \subseteq V$ such that every edge in G is covered by a vertex in S and S is minimized

Constrained COP: *not every subset S is a vertex cover, $\mathcal{F} \neq 2^V$*



Writing MINIMUM VERTEX COVER as UBO form:

QUBO

$$\min_{\mathbf{x} \in \{0,1\}^n} C(\mathbf{x}), C(\mathbf{x}) = \sum_{v \in V} x_v + A \sum_{uv \in E} (1 - x_u - x_v + x_u x_v), A > 1$$

Goal: Penalize uncovered edges

covered: $x_u = 1$ or $x_v = 1$
 $\Rightarrow x_u + x_v - x_u x_v = 1$

uncovered: $1 - x_u - x_v + x_u x_v = 1$



Observed Encoding Gadgets

Choosing exactly one element of a set $\{i, j\}$ consisting of two elements

$$x_i + x_j - 2x_i x_j$$

Choosing none of $\{i, j\}$

$$1 - x_i - x_j + x_i x_j$$

Choosing at least one of $\{i, j\}$

$$x_i + x_j - x_i x_j$$

$A > 1$ ensures that **best $\mathbf{x}$** is minimum vertex cover

MINIMUM VERTEX COVER as QUBO:

$$\min_{\mathbf{x} \in \{0,1\}^n} C(\mathbf{x}), C(\mathbf{x}) = \sum_{v \in V} x_v + A \sum_{uv \in E} (1 - x_u - x_v + x_u x_v), A > 1$$

Why? If $S = \{v \in V \mid x_v = 1\}$ is not a vertex cover: for each edge that is not covered, the contribution to $C(\mathbf{x})$ is A . Consider an uncovered edge uv . Adding u to S will reduce the value of $C(\mathbf{x})$ by $1 - A$

Challenge for VERTEX COVER solutions: Choosing Penalty A

$$\min_{\mathbf{x} \in \{0,1\}^n} C(\mathbf{x}), C(\mathbf{x}) = \sum_{v \in V} x_v + A \sum_{uv \in E} (1 - x_u - x_v + x_u x_v), A > 1$$

Penalty Weight	Feasible Optimal	Feasible	Infeasible	Infeasible
$A = 1.5$	4	5	$3 + 1.5 = 4.5$	$2 + 1.5 * 2 = 5$
$A = 2$	4	5	$3 + 2 = 5$	$2 + 2 * 2 = 6$

Choice of penalties might not support near optimal solutions

- Quantum approaches typically give us *many* solutions
- Solving QUBO quantumly results in many *high probability* but *not only* optimal solutions of penalty-term QUBO
- Non-optimal solutions can be infeasible
- Obtaining infeasible solutions creates unwanted challenges
- Selecting penalty values: hard problem—**instance dependent**
- **Penalties makes sampling quantum circuits less fun**

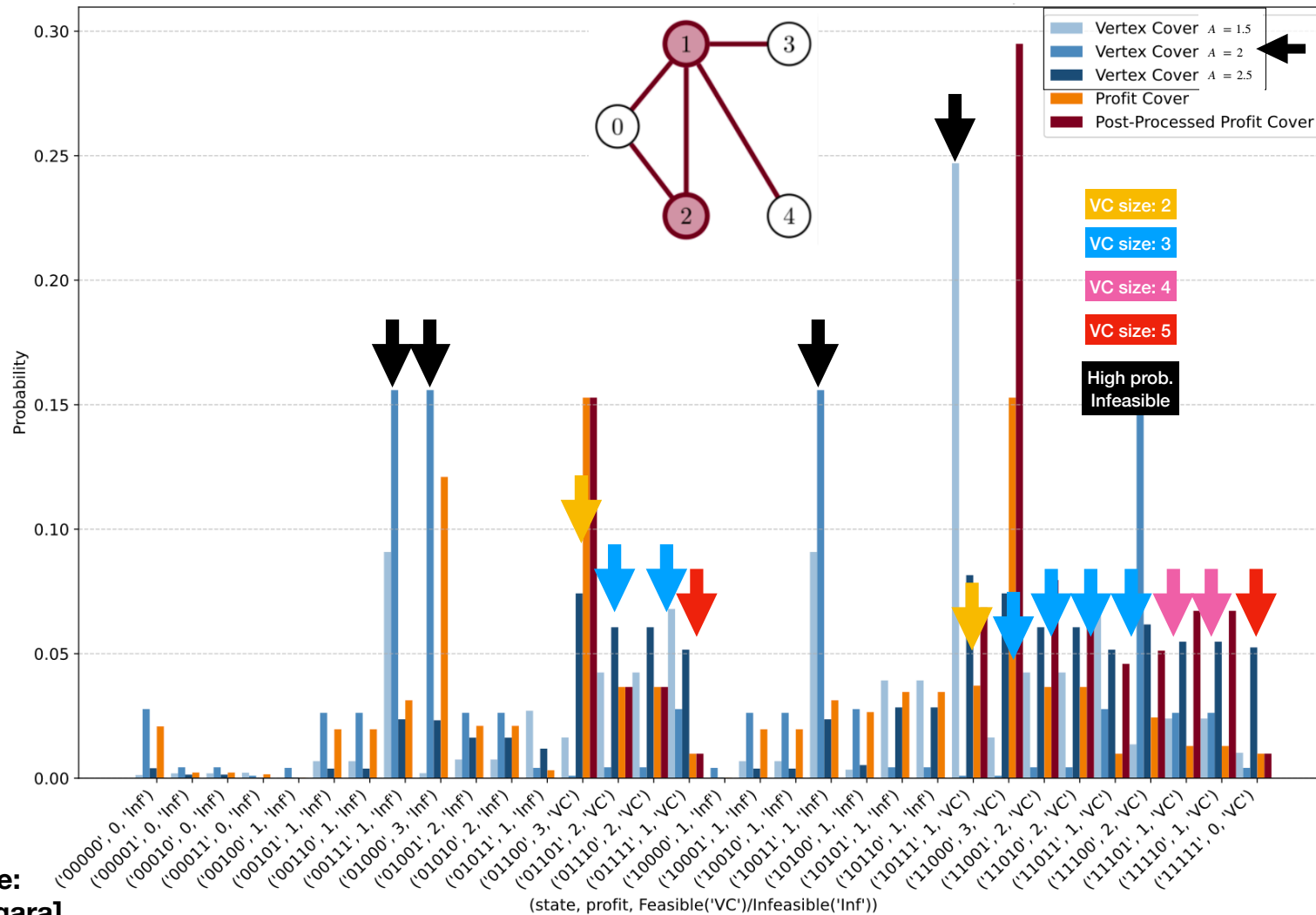


Image source:
PhD Thesis [Angara]

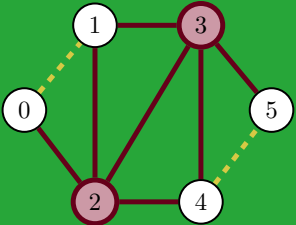
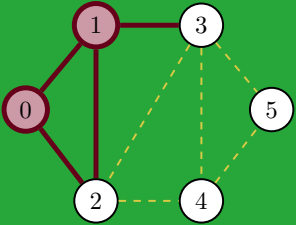
SCOOP Framework—can eliminate constraints

- **Goal:** reduce or eliminate penalty coefficient dependencies while maintaining solution quality
- Example VERTEX COVER

Identify constraint(s)
Every edge must be covered

Measure success / profit
compute $\#(\text{edges covered}) - \#(\text{vertices used})$

Quantify feasibility

	2 vertices used to cover 7 edges
	2 vertices used to cover 4 edges

MAXIMUM PROFIT COVER: an unconstrained COP

MAXIMUM PROFIT COVER

Input: a simple undirected graph $G = (V, E)$

Output: a subset $PC \subseteq V$ with profit $\mathfrak{p}_{PC} = |E_{PC}(G, PC)| - |PC|$

$E_{PC}(G, PC)$: set of edges covered by PC ; we call PC a *profit cover*

Note: Every subset PC of vertices can be evaluated according to its profit $\mathfrak{p}_{PC}$

Relationship between MINIMUM VERTEX COVER and MAXIMUM PROFIT COVER for $G = (V, E)$

- Let $S \subseteq V$ be a vertex cover for G
 - Then $\mathfrak{p}_S = |E_{PC}(G, S)| - |S| = |E| - |S|$
- Let $PC \subseteq V$ be a profit cover for G with $\mathfrak{p}_{PC} = |E_{PC}(G, PC)| - |PC|$
 - If PC **is** a vertex cover then G has a vertex cover of size $|PC| = |E| - \mathfrak{p}_{PC}$
 - If PC **is not** a vertex cover then
 - There is edge $uv \in E$ with $u, v \notin PC$
 - Consider $PC \cup \{u\}$

$G = (V, E)$ has a vertex cover of size at most k iff G has a profit cover with profit at least $|E| - k$

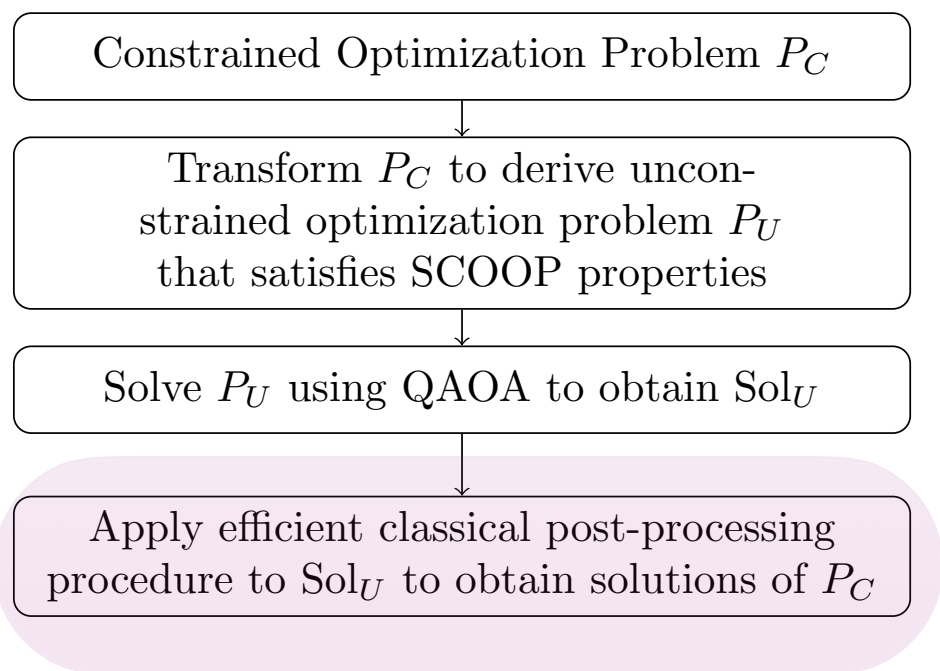
Given maximum profit cover we can derive minimum vertex cover

Vertex cover S can be built with $|S| \leq |E| - \mathfrak{p}_{PC}$

- $\mathfrak{p}_{PC \cup \{u\}} = |E_{PC}(G, PC \cup \{u\})| - |PC \cup \{u\}|$
- $E_{PC}(G, PC \cup \{u\}) \supseteq E_{PC}(G, PC)$
- $|E_{PC}(G, PC)| - |PC| \leq |E_{PC}(G, PC \cup \{u\})| - |PC \cup \{u\}|$

Profit does not get worse!

Pipeline for solving MINIMUM VERTEX COVER using its *unconstrained problem twin* MAXIMUM PROFIT COVER



Algorithm 1 Converting Profit Cover to Vertex Cover

Require: Graph G , Solution PC

Let E_{unc} be the set of uncovered edges in G

for each edge $uv \in E_{\text{unc}}$ **do**

if $u \notin PC$ and $v \notin PC$ **then**

 Add u to PC

end if

end for

UBO for MAXIMUM PROFIT COVER

MAXIMUM PROFIT COVER

Input: a simple undirected graph $G = (V, E)$

Output: a subset $PC \subseteq V$ with profit $\mathfrak{p}_{PC} = |E_{PC}(G, PC)| - |PC|$

Choosing at least one of $\{i, j\}$

$$x_i + x_j - x_i x_j$$

$$C_{PC}(\mathbf{x}) = \sum_{(u,v) \in E} (x_u + x_v - x_u x_v) - \sum_{v \in V} x_v$$

QUBO

$$\min_{\mathbf{x} \in \{0,1\}^n} -C_{PC}(\mathbf{x})$$

Observation

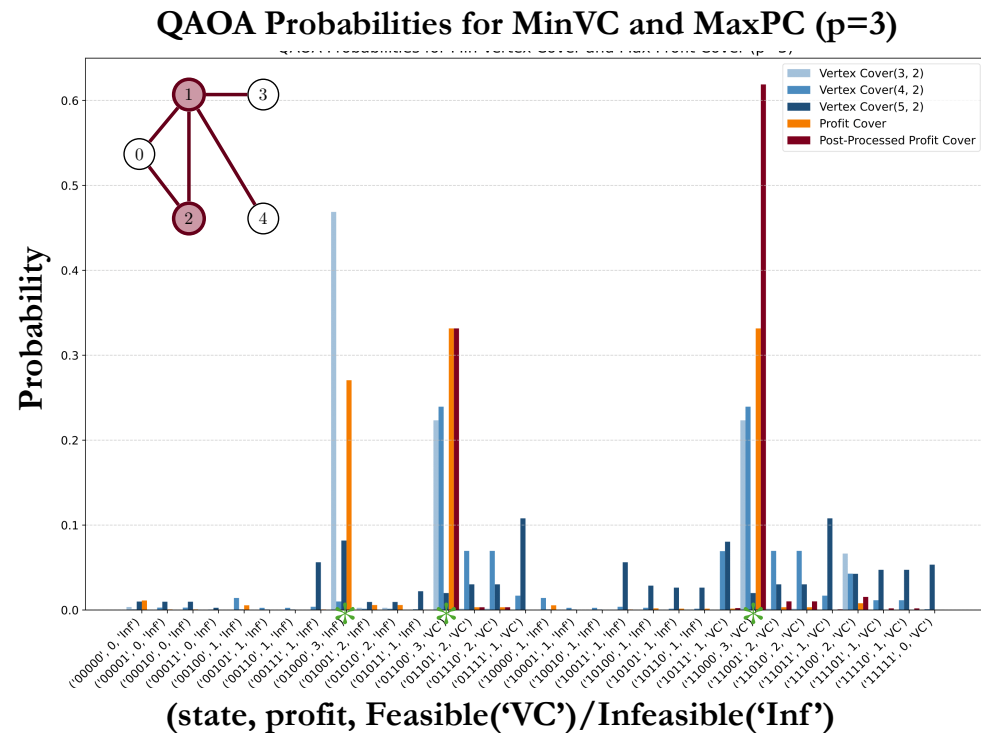
- C_{PC} is quality measure for both: profit covers and vertex covers
- Not true for the cost function of the penalty term QUBO for MINIMUM VERTEX COVER

Probabilities

- Constrained QAOA (blue): Probability of optimal solution < 0.5
- SCOOP QAOA (orange): Higher probabilities for optimal and near-optimal solutions
- Post-processed SCOOP (red): $>90\%$ probability of obtaining correct optimal Vertex Cover

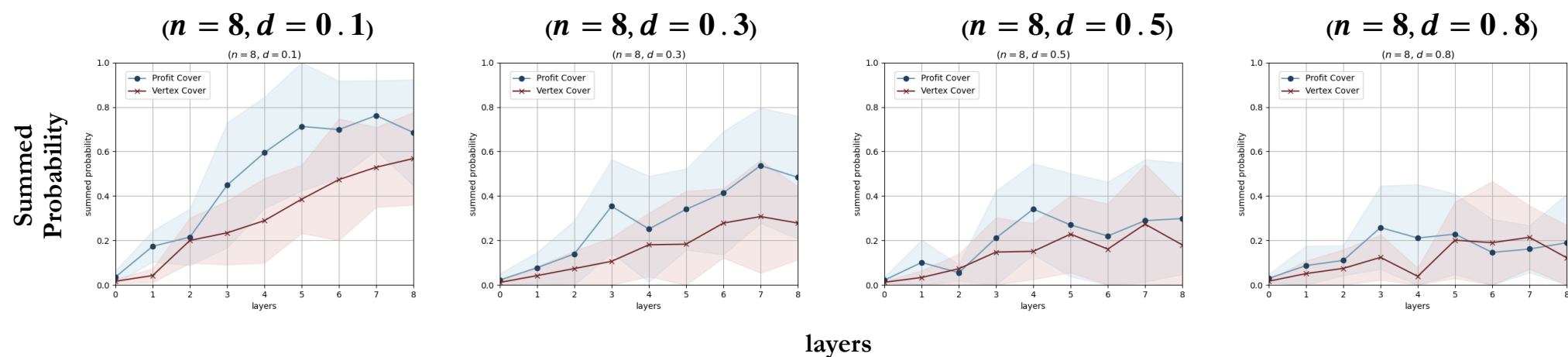
Key takeaway

Encoding a COP using SCOOP dramatically improves solution feasibility and optimality



Summed Optimal Probabilities

● Profit Cover
✕ Vertex Cover

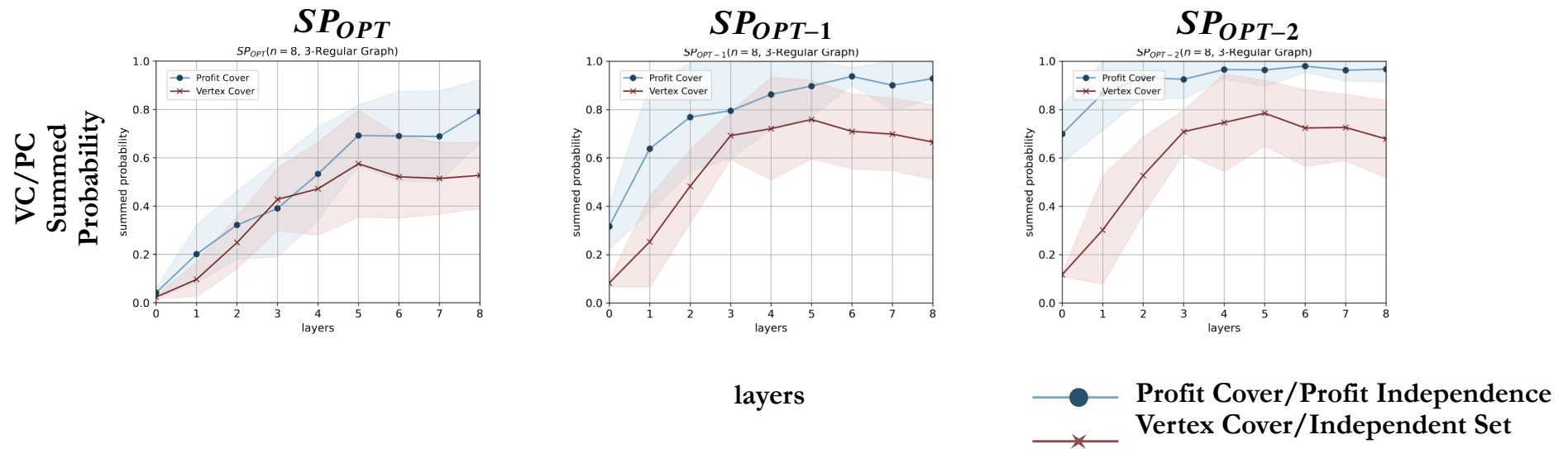


- SCOOP formulations tend to have more optimal solutions than their constrained counterparts.
- Every VC solution of size k maps to PC solution with profit p
- Some infeasible solutions for vertex cover still achieve the same profit; these can be post-processed to obtain feasible vertex covers

Key takeaway

Higher density of optimal solutions
+
the probabilistic nature of quantum
increases the likelihood that QAOA
samples optimal solutions

Summed Near-optimal Probabilities



Total probability of optimal and near-optimal solutions

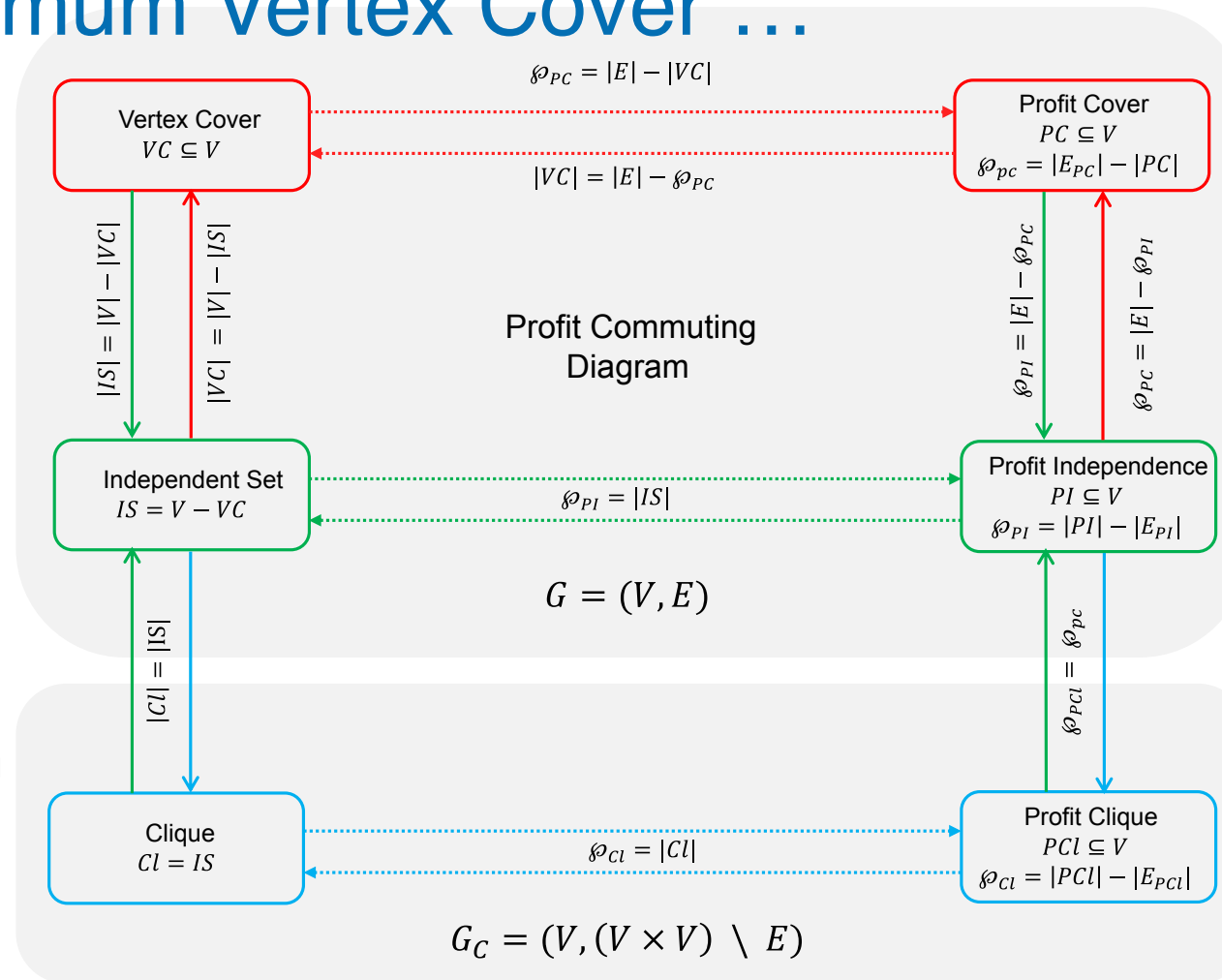
Key takeaway

Even higher density of near-optimal solutions **increases the likelihood that QAOA samples near-optimal solutions.**

Algorithms for Minimum Vertex Cover ...

- Also solve problems
 - Maximum Independent Set
 - Maximum Clique

Maximum Independent Set noted as challenging problem in: Koch et al., “Quantum optimization benchmarking library—the intractable decathlon”, 2025. arXiv:2504.03832



Part 2 — SCOOP

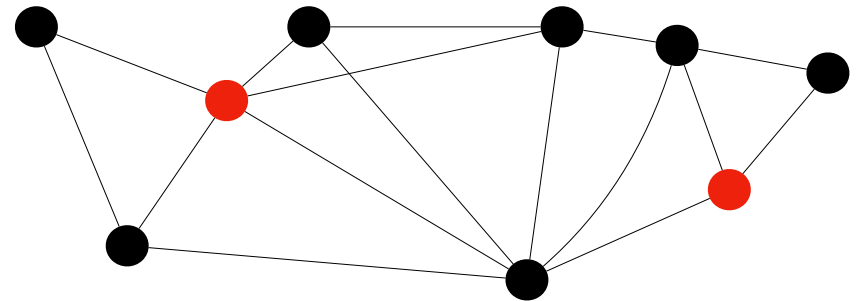
- SCOOP framework
 - Another example: Minimum Dominating set and Profit Domination
 - HUBOs
- From UBOs to cost Hamiltonians to quantum circuits
- Evaluation measures for COPs solved quantumly
- Solving COPs in practice: what to classical and what to quantum? Learning to offload

MINIMUM DOMINATING SET: CCOP

MINIMUM DOMINATING SET

Input: Undirected graph $G = (V, E)$

Output: A smallest subset $V' \subseteq V$ such that for each $v \in V$: $v \in V'$ or $N(v) \cap V' \neq \emptyset$



NP-hard

Constrained combinatorial optimization problem

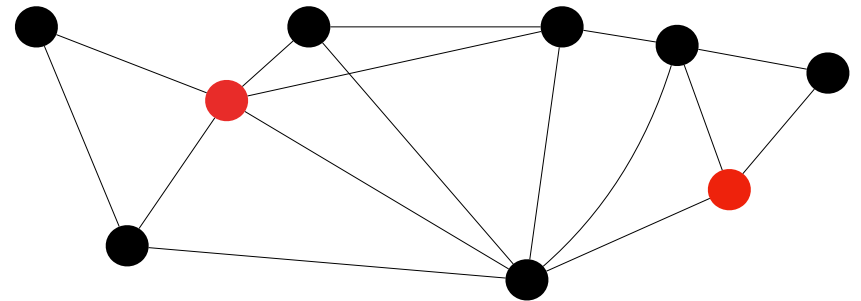
- Not every subset of vertices is a dominating set

A UBO for MINIMUM DOMINATING SET

MINIMUM DOMINATING SET

Input: Undirected graph $G = (V, E)$

Output: A smallest subset $V' \subseteq V$ such that for each $v \in V$: $v \in V'$ or $N(v) \cap V' \neq \emptyset$



For each $i \in V$: introduce binary variable x_i :

- $x_i = 1$ if $i \in V'$,
- $x_i = 0$ otherwise

To be encoded:

- Every vertex is dominated by V' **Constraint**
- The number of vertices in V' is minimized

MINIMUM DOMINATING SET

Input: Undirected graph $G = (V, E)$

Output: A smallest subset $V' \subseteq V$ such that for each $v \in V$: $v \in V'$ or $N(v) \cap V' \neq \emptyset$

A UBO for MINIMUM DOMINATING SET

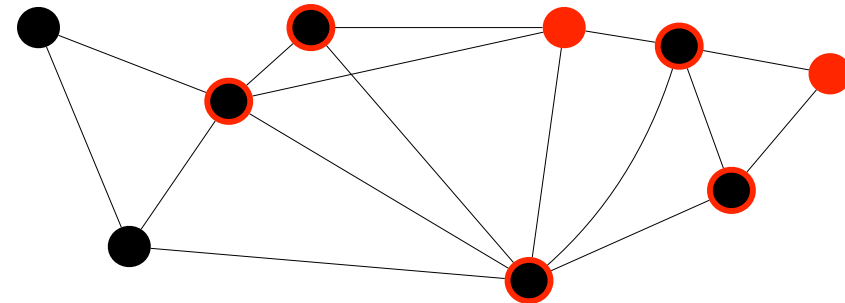
- Every vertex is dominated by V'
 $\text{isDominated}(i, j) = 1$ if $i \in V'$ or $j \in V'$
- The *number of vertices* in V' is minimized

$$\sum_{i \in V} x_i$$

$$x_i = 1 \text{ if } i \in V', x_i = 0 \text{ otherwise}$$

Dominated: Choosing at least one of $\{1, 2, \dots, k\}$

Not dominated: Choosing none of: $\{1, 2, \dots, k\}$



$$(1 - x_i)(1 - x_j)(1 - x_\ell) = \begin{cases} 1 & \text{if } x_i = x_j = x_\ell = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$1 - (1 - x_i)(1 - x_j)(1 - x_\ell) = \begin{cases} 0 & \text{if } x_i = x_j = x_\ell = 0 \\ 1 & \text{otherwise} \end{cases}$$

HUBO for Minimum Dominating Set

$$(1 - x_i)(1 - x_j)(1 - x_\ell) = \begin{cases} 1 & \text{if } x_i = x_j = x_\ell = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$1 - (1 - x_i)(1 - x_j)(1 - x_\ell) = \begin{cases} 0 & \text{if } x_i = x_j = x_\ell = 0 \\ 1 & \text{otherwise} \end{cases}$$

$$\min_{\mathbf{x}} \left(\sum_{i \in V} x_i + A \sum_{i \in V} \left((1 - x_i) \prod_{j \in N(i)} (1 - x_j) \right) \right), A > 1$$

Choosing at least one of $\{1, 2, \dots, k\}$

$$1 - \prod_{i \in \{1, \dots, k\}} (1 - x_i)$$

Choosing none of $\{1, 2, \dots, k\}$

$$\prod_{i \in \{1, \dots, k\}} (1 - x_i)$$

HUBO discourages vertices that are *not* dominated using penalty terms

Observed Encoding Gadgets

Choosing exactly one element of a set $\{i, j\}$ consisting of two elements

$$x_i + x_j - 2x_i x_j$$

Choosing none of $\{i, j\}$

$$1 - x_i - x_j + x_i x_j$$

Choosing at least one of $\{i, j\}$

$$x_i + x_j - x_i x_j$$

Choosing at least one of $\{1, 2, \dots, k\}$

$$1 - \prod_{i \in 1, \dots, k} (1 - x_i)$$

Choosing none of $\{1, 2, \dots, k\}$

$$\prod_{i \in 1, \dots, k} (1 - x_i)$$

A note on HUBOs

- HUBO formulations are not appropriate for Quantum Annealing (DWave)
- The higher the order of the HUBO, the more computational intensive it is to solve the problem
- HUBOs can be converted to QUBOs or to lower order HUBOs, normally requires the introduction of (additional) penalty terms [Glover et al.]
- Useful technique: Rosenberg quadratization, see <https://doi.org/10.1051/ro/197206V200951>

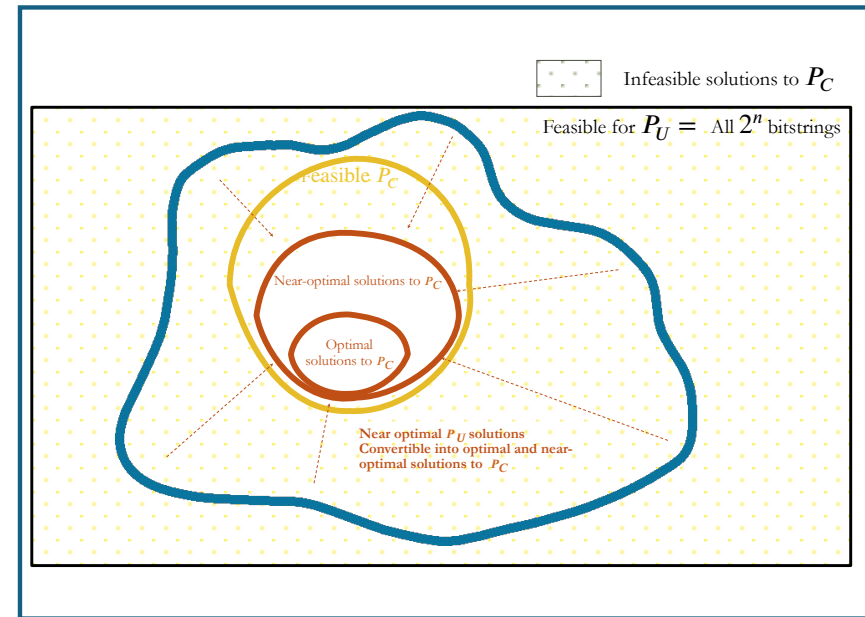
SCOOP: Unconstrained problem twin for DOMINATING SET?

SCOOP Framework: Quantum-ready formulations for constrained problems

SCOOP twins are solution-enhanceable, constrained-unconstrained, objective-function-compatible optimization problems

COP P_C and UCOP P_U are SCOOP twins if:

- 1. Identical Input Domains:** P_C and P_U take the same kind of input
- 2. Solution Containment:** Every feasible solution to P_C is a solution for P_U
- 3. Objective function compatibility:** Solutions can be translated between problems in polynomial time
- 4. Solution Enhance-ability:**
 - A solution to P_U satisfies constraints of P_C
 - Or a solution to can be effectively corrected to a feasible solution to P_C with equivalent or better objective value



Guidelines to find SCOOP twins

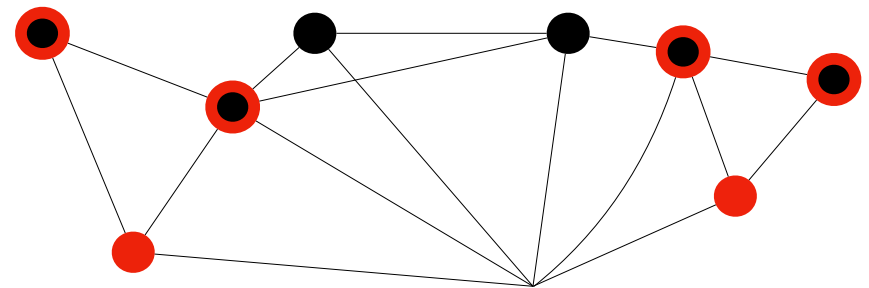
1. Identify constraints
2. Quantify feasibility of infeasible solution
3. Develop unconstrained objective function compatible with constrained objective
4. Develop classical solution post-processing to transform any solution to the unconstrained problem to a solution for the constrained problem while maintaining or improving solution quality

Deriving an unconstrained problem twin for DOMINATING SET

- Identify constraint(s): Every vertex must be dominated
- Quantify feasibility: 6 vertices dominated using two chosen vertices
- Measure success / profit of dominating vertices through a subset $PD \subseteq V$ by:

$$|\{v \in V \mid v \text{ is dominated by } PD\}| - |PD|$$

- Note: we still must prove that solution quality is preserved



MINIMUM DOMINATING SET

Input: Undirected graph $G = (V, E)$

Output: A smallest subset $V' \subseteq V$ such that for each $v \in V$: $v \in V'$ or $N(v) \cap V' \neq \emptyset$

Relationship between dominating sets and profit domination sets

MAXIMUM PROFIT DOMINATION

Input: Undirected graph $G = (V, E)$

Output: A subset $PD \subseteq V$ such that $\mathfrak{p}_{PD} = |\{v \in V \mid v \text{ is dominated by } PD\}| - |PD|$ is maximized

Algorithm 1 Converting solutions: MAXPD to MINDS
adapted from [38]

Require: Graph G , Solution PD

Let V_{nd} be the set of non-dominated vertices in G

for each vertex $v \in V_{\text{nd}}$ **do**

if $N(v) \not\subseteq PD$ and $v \notin PD$ **then**

 Add v to PD

end if

end for

For any singleton-free graph $G = (V, E)$, G has a dominating set $DS \subseteq V$ of size k if and only if G has a subset $PD \subseteq V$ with profit $\mathfrak{p}_{PD} = |V| - k$.

Summary of SCOOP Twins

	CCOP	SCOOP	Constraint Avoided	Encoding	Complexity
Vertex Cover	MinVC	MaxPC	Edge coverage	QUBO	NP-Hard
Independent Set	MaxIS	MaxPI	Vertex independence	QUBO	NP-Hard
Clique	MaxCI	MaxPCI	Completeness of subgraph	QUBO	NP-Hard
Set Packing	MaxSP	MaxPSP	Set disjointedness	QUBO	NP-Hard
3 SAT	Max3SAT*	MaxPI*	Quadratization constraint	QUBO	NP-Hard
Dominating Set	MinDS	MaxPD	Vertex domination	HUBO	NP-Hard
two con- straints	Maximal Matching	Min Max Matching	MaxPES	HUBO	NP-Hard
EdgeDominating Set	MinEDS	MaxPES	Edge domination	HUBO	NP-Hard
Independent EdgeDominating Set	MinIEDS	MaxPES	Edge domination, independence	HUBO	NP-Hard
Set Cover	MinSC	MaxPSC	Set coverage	HUBO	NP-Hard
Edge Cover	MinEC	MaxPEC	Edge coverage	HUBO	P
Matching	MaxM	MaxPEC	Matching	HUBO	P

Angara, PhD thesis, 2025

Next: From UBO to Quantum Gate

Encoding UBOs with Quantum Gates

- **Step 1**
 - Transform QUBOs into *classical Ising models* or
 - Transform HUBOs into *higher order spin models*
- **Step 2** Build *cost Hamiltonian*
- **Step 3** Determine unitary matrix for cost Hamiltonian
- **Step 4** Build a gate sequence for unitary

Goal: encode cost function as input to QAOA

→ find H_c such that
 $H_c |\mathbf{x}\rangle = C(\mathbf{x}) |\mathbf{x}\rangle$

Ising models and higher order spin models

- **Recall:** QUBO $f_Q(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n Q_{ij} x_i x_j = \mathbf{x}^\dagger Q \mathbf{x}$ for $x_i, x_j \in \{0,1\}$
- *Spin model* describes interaction of particles (or *spins*) with each other in lattice/network structure
 - Each node i represents *spin* z_i (states: $+1$ or -1)
- Quadratic spin (aka Ising) model: $H = -J \sum_{(i,j)} z_i z_j - \sum_i h_i z_i$
 - J : constant reflecting strength of interaction
 - h_i : external magnetic field acting on spin z_i

Transforming QUBO into Ising formulation

- Given QUBO $f_Q(\mathbf{x}) = \mathbf{x}^\dagger Q \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n Q_{ij} x_i x_j$
 - **Ising/Spin model:** instead of *binary* vector $\mathbf{x}$, here the elements of vector $\mathbf{z}$ each have value $z_i \in \{-1, 1\}$
 - Transformation: $x_i := \frac{1 - z_i}{2}$
ie: $z_i = 1 \leftrightarrow x_i = 0$ and $z_i = -1 \leftrightarrow x_i = 1$

Deriving Ising formulation for MAXCUT

QUBO cost function $\sum_{(i,j) \in E} 2x_i x_j - x_i - x_j$, for $x_i := \frac{1 - z_i}{2}$

Ising formulation: $\sum_{(i,j) \in E} \left(2 \frac{1 - z_i}{2} \frac{1 - z_j}{2} - \frac{1 - z_i}{2} - \frac{1 - z_j}{2} \right)$

$$= \frac{1}{2} \sum_{(i,j) \in E} \left(2(1 - z_i)(1 - z_j) - (1 - z_i) - (1 - z_j) \right)$$

$$= \frac{1}{2} \sum_{(i,j) \in E} \left(2 - 2z_i - 2z_j + 2z_i z_j - 1 + z_i - 1 + z_j \right) = \frac{1}{2} \sum_{(i,j) \in E} \left(2z_i z_j - z_i - z_j \right)$$

Where $\frac{1}{2} \sum_{(i,j) \in E} \left(2z_i z_j - z_i - z_j \right)$ is to be minimized

In general: Transforming QUBO into Ising form

For $f_Q(\mathbf{x}) = \mathbf{x}^\dagger Q \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n Q_{ij} x_i x_j$ and $x_i := \frac{1 - z_i}{2}$ we obtain

$$f_Q(\mathbf{z}) = \sum_{i=1}^n \sum_{j=1}^n Q_{ij} \frac{1 - z_i}{2} \frac{1 - z_j}{2} = \frac{1}{4} \sum_{i=1}^n \sum_{j=1}^n Q_{ij} (1 - z_i)(1 - z_j)$$

Ising Form to Cost Hamiltonian

Given $f_Q(\mathbf{z}) = \frac{1}{4} \sum_{i=1}^n \sum_{j=1}^n Q_{ij}(1 - z_i)(1 - z_j)$

Note:

$$(I - Z_i)|z\rangle = (1 - z_i)|z\rangle$$

Think: each element of $\mathbf{z}$ corresponds to qubit:

cost Hamiltonian $H_C = \frac{1}{4} \sum_{i=1}^n \sum_{j=1}^n Q_{ij}(I - Z_i)(I - Z_j)$

Here: Z_i is Pauli-Z applied to i -th qubit, I is identity matrix

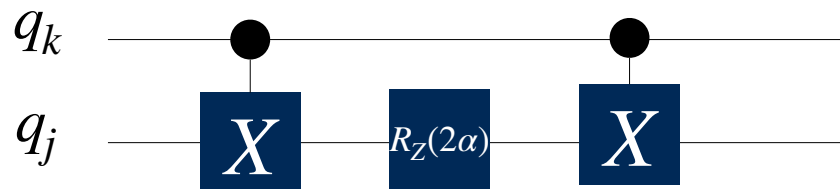
H_C Hermitian (not necessarily unitary)

What is a gate sequence for e^{-itH_C} ?

- Consider

$$H_C = Z \otimes Z \otimes I + I \otimes Z \otimes Z - \frac{1}{2}Z \otimes I \otimes I - I \otimes Z \otimes I - \frac{1}{2}I \otimes I \otimes Z = Z_1Z_2 + Z_2Z_3 - \frac{1}{2}Z_1 - Z_2 - \frac{1}{2}Z_3$$

- Then $e^{-i\alpha H_C} = e^{-i\alpha Z_1Z_2} e^{-i\alpha Z_2Z_3} e^{i\frac{\alpha}{2}Z_1} e^{i\alpha Z_2} e^{i\frac{\alpha}{2}Z_3}$
- Implement via Z -rotation gate:
 - $e^{-i\alpha Z} : \boxed{R_Z(2\alpha)}$ — rotates state around Z -axis of Bloch sphere by 2α
 - $e^{-i\alpha Z_kZ_j} :$



QAOA with higher-order encodings

Gate-based vs QA Hardware

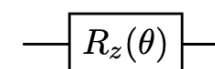
- QA: strictly limited to QUBO formulations
- Gate-based: supports higher-order terms via multi-qubit gates

HUBO (Higher-Order Unconstrained Binary Optimization)

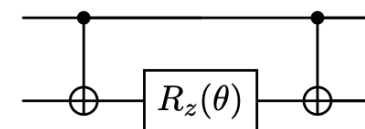
- More expressive than QUBO
- Encodes multi-qubit interactions with multi-qubit gates
- Increasingly feasible with advances in fidelity & connectivity of gate-based quantum computers

Quadratization Trade-off

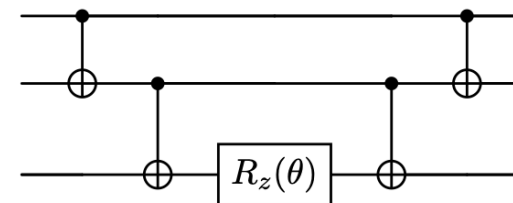
- HUBO to QUBO requires auxiliary variables & penalty terms
- Simpler, but increases # qubits and complexity



(a) $\exp(-i\frac{\theta}{2}Z)$



(b) $\exp(-i\frac{\theta}{2}Z \otimes Z)$



(c) $\exp(-i\frac{\theta}{2}Z \otimes Z \otimes Z)$

Note on Quadratzation vs. HUBOs

- Sparsity mismatch: A problem sparse in Ising space may be dense in Boolean space and vice versa
- Degree reduction in Ising space: Mandal et al. propose methods that reduce polynomial degrees directly in the Ising formulation
- There is a trade-off between qubit count and circuit depth for efficient binary encodings

Insight

Feasibility of encoding HUBOs depends on problem structure; understanding structural properties is crucial for determining the necessity of higher-order terms.

Mandal, A. et al. (2020). Compressed quadratzation of higher order binary optimization problems. In Proc. 17th Proc. ACM Int. Conf. on Computing Frontiers

Implementing the Mixer

For standard mixer $H_M = \sum_{j=1}^n X_j$, the unitary is $\prod_{j=1}^n e^{-i\beta X_j}$

Use X -rotation gate $R_X(\theta) = e^{-i\theta X/2}$

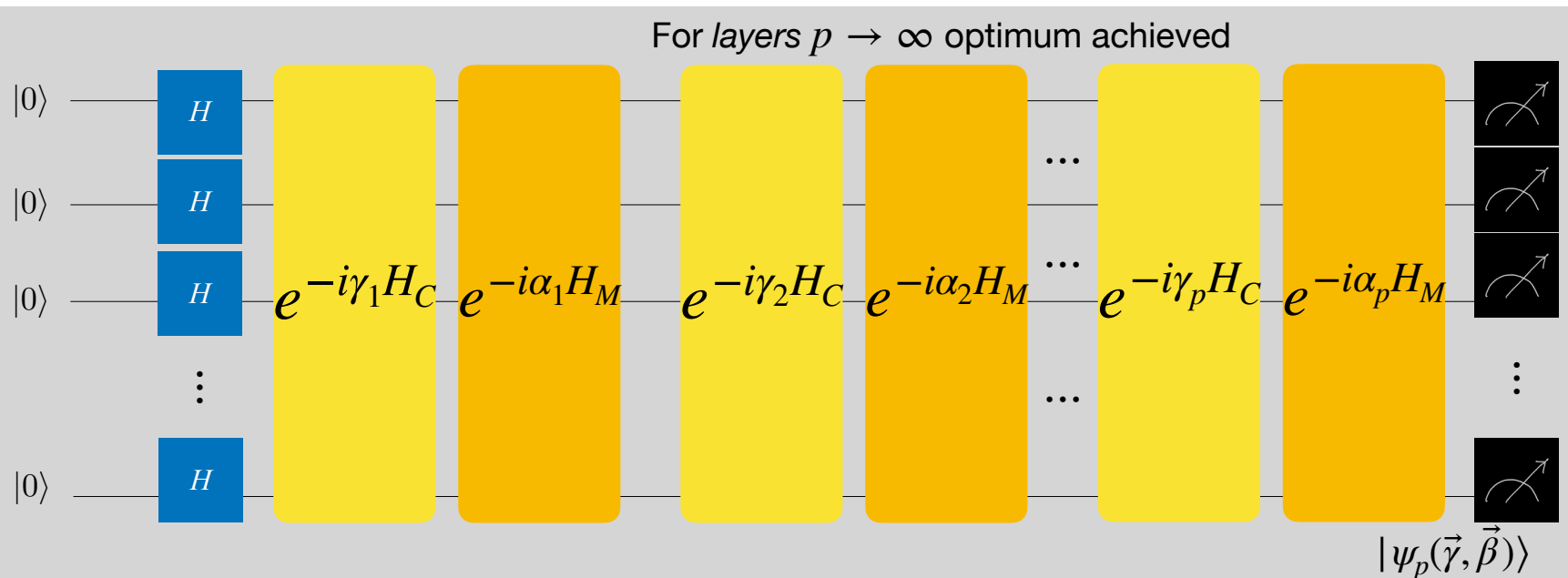
Then $e^{-i\beta X} = R_X(2\beta)$; each $e^{-i\beta X_j}$: $R_X(2\beta)$ -gate on qubit j

Gate sequence can be executed in parallel (no 2-qubit gates)

You are ready to run it!

QAOA: Variational Quantum Algorithm

Alternating Ansatz



H_C : cost Hamiltonian, encodes objective function
 H_M : mixer Hamiltonian

γ_j : parameter “how strong” cost function is applied
 α_j : “push strength” helps keep from getting stuck

Classical
Optimizer

$$\langle \psi | H_C | \psi \rangle$$

$$(\vec{\beta}, \vec{\gamma})$$

Goal:
 High
 probability /
 good quality
 solution

A note on evaluating experiments

- For constraint problems the original proposed measure (eg solution size as for Vertex Cover and Dominating Set) may not reflect the cost function derived when containing penalty terms
- The cost function for the unconstrained problem twin is generally a better way to evaluate and compare obtained solutions

Evaluation Measures

- **Probabilities:** Likelihood of observing specific solutions
- **Summed Probabilities:** Total probability of optimal and/or near-optimal solutions
- **Expectation Values:** Average cost or objective over the obtained quantum state
- **Approximation Ratios:** Ratio of solution quality to the optimal solution $\frac{\langle \psi | H_C | \psi \rangle}{\text{Optimal Cost}}$

Different objectives have widely different scales

Constrained vs. unconstrained objectives cannot be directly compared

Expectation value interpretation varies:

- SCOOP: expectation value = profit → clear measure of solution quality
- Constrained problems: expectation value alone does not indicate solution quality

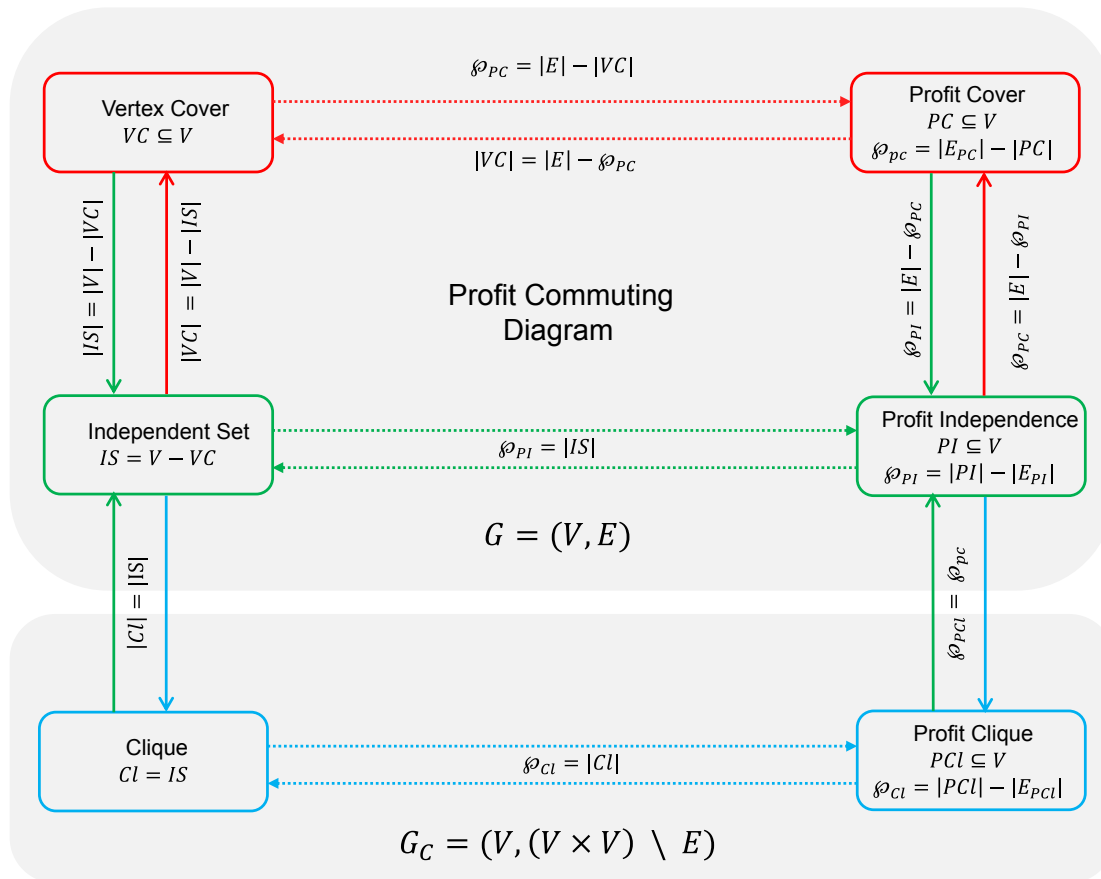
Approximation Ratio Limitations for penalty-based encodings

- Works well for unconstrained problems with natural QUBO/HUBO encodings, where cost values reflect solution quality
- Fails for penalty-based encodings of constrained problems: near-optimal solutions may have inflated or misleading costs
- Example: QAOA returns a high-quality feasible solution (25%) and a lower-quality infeasible solution (50%); poor penalty settings may make the infeasible solution appear better, skewing the approximation ratio
- Expectation value simulators (like QTensor) might not be useful to draw conclusions: they average over all solutions, including infeasible ones, hiding the true solution probabilities

Insight

Approximation ratio can be misleading for penalty-encoded problems, especially when near-optimal solutions matter and penalty parameters are not perfectly tuned.

Making practical use of classical results



Build Pipeline

- *Solve or pre-process* what classical methods can do efficiently and exactly
- *Offload* remaining hard instance to quantum hardware
- *Post-process* classically and efficiently

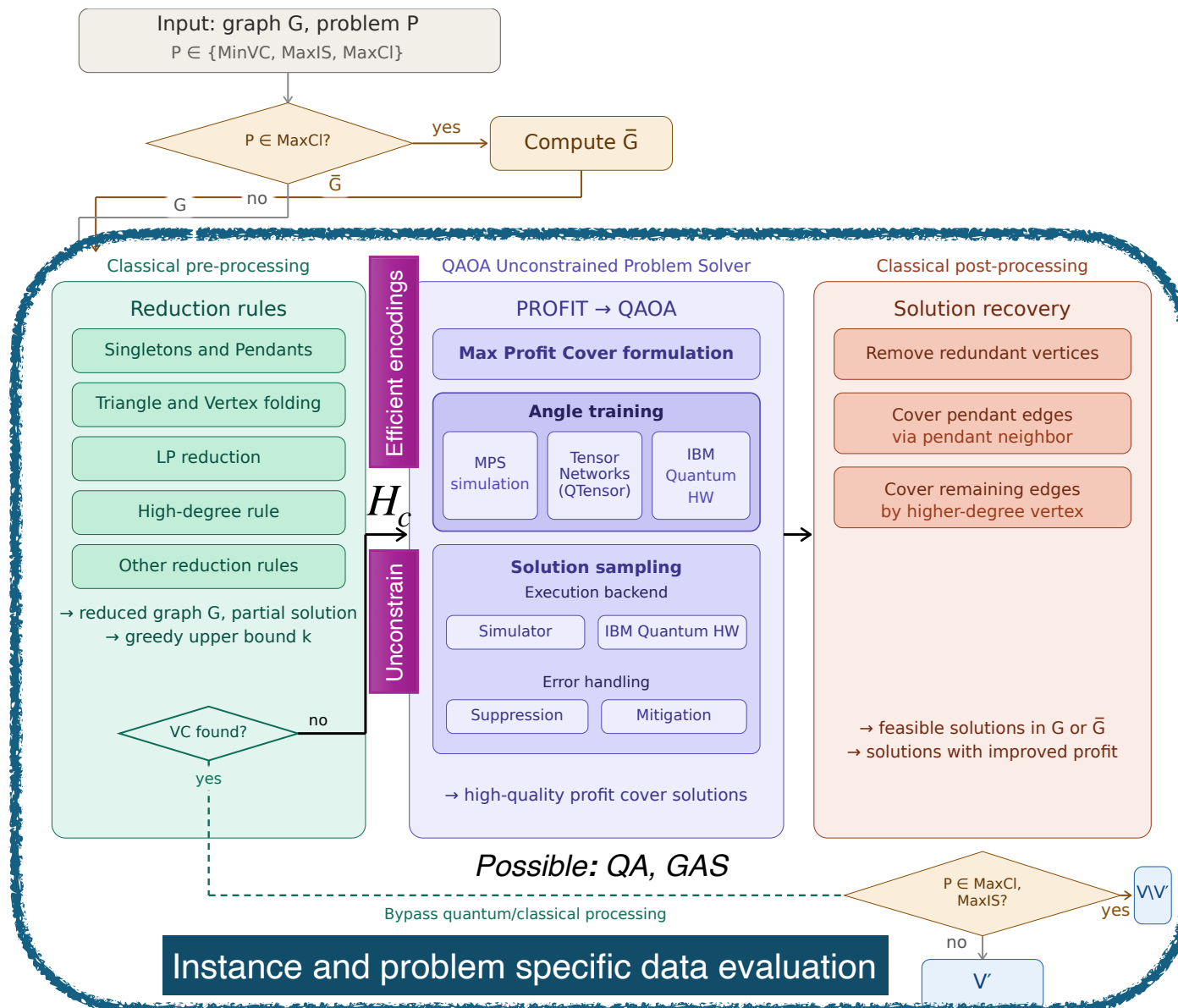
Systematic investigation of instance qualities with respect to classical and quantum solvability

Angara et al., arXiv:2604.25162

Classical pre-processing: Reducing problem size/transformations classical methods; quickly and exactly

QAOA Unconstrained Problem Solver
Offload to quantum hardware only what cannot be handled fast classically

Classical pre-processing:
Solution recovery



Some Results: using IBM Quebec Heron r2 processor and post-processing

QAOA: 1 layer											
Name	V	E	MaxIS/ MinVC	MaxPC Profit	2Q Depth	Non-local Gates	$\approx$ Ratio	Profit	Post- proc. IS	Post-proc. $\approx$ Ratio	Refined Post-Proc. Profit
farm	17	39	10 (7)	32	89	174	0.300	25	10	1.000	32
mammalia- kangaroo- interactions	17	91	4/13	78	118	367	0.250	75	4	1.000	78
ibm32	32	90	13/19	71	175	528	0.385	63	9	0.692	67
karate	34	78	20/14	64	202	426	0.250	49	20	1.000	64
football	35	118	16/19	99	258	784	0.312	88	13	0.812	96
chesapeake	39	170	17/22	148	353	1076	0.176	134	15	0.882	146
aves-sparrow- social	52	454	13/39	415	808	2842	0.308	406	13	1.000	415
insecta-ant- colony1-day38	56	1134	6/50	1084	3168	8994	0.500	1081	5	0.833	1083
sloane_1dc_64	64	543	10/54	489	1114	3873	0.100	480	8	0.800	487
es60fst03	113	142	55/58	84	125	799	0.509	57	50	0.909	79
es60fst01	123	159	60/53	96	168	954	0.533	68	58	0.967	94
C125.9	125	787	34/92	696	1469	10507	0.294	672	29	0.853	691
sloane_1zc_128	128	1120	18/110	1010	1816	11659	0.056	993	16	0.889	1008
sloane_1dc_128	128	1471	16/112	1359	2556	13555	0.125	1345	13	0.812	1356

Example of End-to-end results on real-world instances (networkrepository.com)

Graph				Classical Pre-Proc.		QAOA Solver	Classical Post-Proc.			
Name	$ V $	$ E $	$ \bar{E} $	$ V_{\text{safe}} $	$G'(V, E)$ or $\bar{G}'(V, \bar{E})$	Best p_{PC}	p_{PC}	$\mathcal{P}$	$ \text{Sol} $	$ \text{Sol} _{\text{known}}$
	⋮				⋮		⋮			
	insecta-ant-colony1-day38	56	1134		(53, 1048)	969	1000	MAXIS	6	6
	insecta-ant-colony1-day38	56		406	(47, 210)	178	185	MAXCL	24	22
	⋮				⋮		⋮			
	ca-netscience	379	914		(86, 195)	126	130	MINVC	238	214
	ca-netscience	379		70717	Solved	Bypassed	Bypassed	MAXCL	9	9

Angara et al., “Experimental Workflows for Combinatorial Optimization: Towards Quantum Advantage”, 2026 arXiv:2604.25162

Practice materials: <https://github.com/pangara/profit-summer-school-26>

Jupyter notebook: Lab.ipynb

Simulator or IBM hardware

- Learning Objectives: by the end you should be able to
 - Represent a graph optimization problem as a Hamiltonian
 - Construct a QAOA circuit for constrained optimization problems
 - Understand the role of cost and mixer operators
 - Optimize QAOA parameters using a classical optimizer
 - Sample candidate solutions from a quantum circuit
 - Verify whether a sampled solution satisfies the problem constraints
 - Interpret the quality of solutions produced by QAOA

Practice materials: <https://github.com/pangara/profit-summer-school-26>

Local Python files

hamiltonian.py

- Constructing problem Hamiltonians
- Building QAOA cost and mixer operators
- Generating QAOA circuits

qaoa.py

- Constructing problem Hamiltonians
- Building QAOA cost and mixer operators
- Generating QAOA circuits

utils.py

- Graph visualization
- Solution validation including post processing
- Plotting and analysis utilities

Workflow

1. Create a graph optimization problem
2. Convert the problem into a quantum cost Hamiltonian
3. Build a QAOA ansatz
4. Optimize the QAOA parameters
5. Sample candidate solutions from the optimized circuit
6. Verify constraint satisfaction
7. Visualize and interpret the resulting solution

Useful Readings

Prashanti Angara, Danylo Lykov, Ulrike Stege, Yuri Alexeev, Hausi Müller, “The Art of Avoiding Constraints: A Penalty-free Approach to Constrained Combinatorial Optimization with QAOA”, 2025. Submitted to TQE, available on arXiv:2503.10077

Prashanti Angara, Emily Martins, Ulrike Stege, Hausi Müller, “SCOOP: A Quantum-Computing Framework for Constrained Combinatorial Optimization”, QCE 2025, available on arXiv:2504.10897

Prashanti Angara. “Flexible integration of classical and quantum techniques in the evolutionary path to quantum utility”, PhD Dissertation, 2025, University of Victoria

Prashanti Angara, Felipe Rivera, Ibrahim Shehzad, Ulrike Stege, Hausi Müller, “Experimental workflows for combinatorial optimization: Towards quantum advantage”, available on arXiv:2604.25162

Koch et al., “Quantum optimization benchmarking library — the intractable decathlon”, 2025. arXiv:2504.03832

Lucas, “Ising Formulations of many NP Problems”, 2014, arxiv:1302.5843

Glover, F., Kochenberger, G., Hennig, R., and Du, Y. (2022). Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Annals of Operations Research*, 314(1):141–183.

Summary

- SCOOP method for hard CCOPs
- Workflow
 - Classic pre-processing
 - Encoding using SCOOP transformation
 - Solve quantumly using QAOA
 - Classic post-processing
- Extensive Experiments
 - Decathlon and networkrepository.com problem sets
 - Simulators, IBM Heron
- Workbook

Thank you!

Thanks to
Priya Angara, Tristan
Zaborniak, Hausi
Müller, Emily Martins:
UVic;
Yuri Alexeev, Danylo
Lykov: NVIDIA;
Ibrahim Shehzad, Sean
Wagner: IBM

